Lighthouse

Building an agent-guided mobile robot from an empty Gazebo world — a design textbook

Dan · with Hermes

2026-10-26

Contents

1 — Preface **4**
What "verified" means in this book . . . 4
How the book is organised . . . 4
Three paths through the book . . . 5

The system at a glance **6**
1.1 The task . . . 6
1.2 Three phases, three additions . . . 6
1.3 Five layers, and the one rule that keeps them apart . . . 7
1.4 What the robot can do today . . . 7
1.5 Acceptance at a glance . . . 8

2 — Eight design principles **9**
2.1 Simulation is a measuring instrument, not a toy . . . 9
2.2 One source of truth for the world . . . 9
2.3 The model decides what, code decides whether and how . . . 9
2.4 One 2D contract in the middle . . . 10
2.5 Freeze what passed; add next to it . . . 10
2.6 Measure with one decoder, and never trust a single map statistic . . . 10
2.7 One run per arm is not an experiment . . . 10
2.8 Fail loudly at startup, report honestly at the end . . . 11

3 — The house: world generation **12**
3.1 Why the world is code . . . 12
3.2 Geometry in one table . . . 12
3.3 Furniture, and the two-height trick . . . 14
3.4 The corridor landmarks, and why the label does not match the position . . . 14
3.5 Self-checks that run with the generator . . . 14

4 — The robot: Lighthouse **16**
4.1 One robot, two sensor heads . . . 16
4.2 Chassis numbers that the rest of the stack depends on . . . 16
4.3 The TF tree is a contract . . . 16
4.4 Four hardware bugs worth remembering . . . 17
4.5 Sensors the agent layer can use . . . 17

5 — The runtime: one launch file, fifteen switches **19**
5.1 Why one launch file . . . 19
5.2 The order matters: a timetable, not a list . . . 20
5.3 The bridge contract . . . 20
5.4 Reality check before every launch . . . 20
5.5 Isolation: how experiments avoid touching the live map . . . 21

6 — The laser contract: from photons to /scan **22**
6.1 Two sensors, one message . . . 22
6.2 Why a 2D slice, and what it costs . . . 23
6.3 The failure that motivated an assertion . . . 23
6.4 The guard: one publisher, and it must be the right one . . . 23

7 — SLAM: building the map that everything else trusts **25**
7.1 Configuration . . . 25
7.2 What the map looks like, quantitatively . . . 25
7.3 The ghosting investigation, and one retracted conclusion . . . 26
7.4 Localisation: three modes, one unfinished branch . . . 26
7.5 Map hygiene: checkpoints, preservation, naming . . . 26
7.6 Known SLAM weaknesses . . . 27

8 — Nav2: making a 0.42 m robot fit a 1.3 m door **28**
8.1 Four parameter files, four jobs . . . 28
8.2 The numbers that decide feasibility . . . 28
8.3 Where the costmap comes from . . . 29
8.4 Why the controller "decides" to stand still . . . 29
8.5 Behaviour tree . . . 29
8.6 Progress checker, and what it really measures . . . 30

9 — Two ways to map, and how to measure them **31**
9.1 Tour versus exploration . . . 31
9.2 The exploration node, in the order it makes decisions . . . 31
9.3 What was actually compared . . . 32
9.4 Why no configuration beat the test map . . . 34
9.5 The driver: how to run an experiment instead of a demo . . . 34
9.6 Methodology: what this chapter cost . . . 34

10 — The agent layer: language in, motion out **36**
10.1 Four processes, two Python environments . . . 36
10.2 The /agent/* contract . . . 36
10.3 Models: measured, not evangelised . . . 37
10.4 Why the model is not allowed to decide to move . . . 37
10.5 The tool surface . . . 38
10.6 Three self-checks after every run . . . 38

11 — The semantic map: a database, not a perception system **39**
11.1 What it is . . . 39
11.2 The lexicon, and how a phrase becomes a candidate . . . 39
11.3 Standoff: where a robot should stand to interact with a thing . . . 40
11.4 The observation pose, and why 0.55 m fails . . . 40
11.5 Mode D: vision confirmation with closed options . . . 40
11.6 Mode E: passability, where the camera is genuinely useful . . . 41
11.7 What the semantic map does not do . . . 41

12 — Changing its mind: preemption **42**
12.1 The interaction contract . . . 42
12.2 Judgment order is a code decision . . . 42
12.3 Three pieces, all required . . . 42
12.4 Four bugs that only a live run could find . . . 43
12.5 Acceptance evidence . . . 43
12.6 What remains fragile . . . 44

13 — How we know it works **45**

13.1 Test layers . . . 45
13.2 The offline suite . . . 45
13.3 The eight runtime gates . . . 46
13.4 Isolation, or why the numbers can be trusted . . . 47
13.5 The scan-guard scenarios (reproducible in five minutes) . . . 47
13.6 Six rules of evidence used throughout . . . 47

14 — Delivery: what is frozen, what is open **49**
14.1 The lineage of every artefact . . . 49
14.2 The frozen snapshot . . . 50
14.3 Reproducing the system from zero . . . 50
14.4 What is deliberately not done . . . 51
14.5 Reading order for a newcomer . . . 51

15 — From a house to a warehouse **52**
15.1 What is world-specific today . . . 52
15.2 What is hardcoded that would have to change . . . 52
15.3 Three changes that are architectural, not configurational . . . 53
15.4 Effort, in self-contained steps . . . 53
15.5 The recommendation recorded here . . . 54
15.6 Decision . . . 54

Appendix A — Topics and nodes **55**
A.1 Topics . . . 55
A.2 Nodes . . . 55

Appendix B — Parameter reference **57**
B.1 Localisation and TF . . . 57
B.2 SLAM . . . 57
B.3 Scan projection (3D mode) . . . 57
B.4 Navigation . . . 57
B.5 Exploration . . . 58
B.6 Agent layer . . . 58

Appendix C — Glossary (Chinese ↔ English) **59**

Appendix D — File index **60**
D.2 Configuration and data . . . 61

Appendix E — Experiment log **62**

1 — Preface

This book documents one project end to end: a differential-drive robot called **Lighthouse** that maps a seven-room house in the Gazebo Fortress simulator, navigates it with Nav2, explores it on its own, and accepts instructions in natural language from a local LLM agent — including the ability to change its mind mid-drive.

It is written for someone who has to take over this project: a new student, a colleague, or you in six months. That reader does not need a tour of the code. They need to know **why each piece exists, which piece owns which decision, where the truth lives, and what has actually been measured versus what is merely believed.** Those four things are the structure of every chapter.

What "verified" means in this book

Every number quoted here was produced by running something and reading its output; nothing is "should work". Where a claim is a design intention rather than a measurement, it is marked as such. Where the project deliberately does not do something, the omission is documented in Chapter 14 rather than hidden — the honest gaps are: localisation with AMCL on a live map has never been re-accepted since the truth-pose shortcut was adopted, the EKF drift over long runs is measured but unsolved, and the 2D costmap cannot see obstacles above the laser slice.

How the book is organised

Part	Chapters	Question it answers
I — Orientation	1–2	What does the system do, and by which rules is it built?
II — The world	3–5	Where does the simulated house, the robot model and the runtime come from?
III — The classical stack	6–9	How do laser data, SLAM, Nav2 and exploration actually work here?
IV — The agent	10–12	How does a language model drive this robot without being allowed to lie?
V — Engineering	13–15	How do we know it works, what is frozen, what is still open?

Appendix A lists every topic and node, Appendix B every tuning parameter, Appendix C a Chi-

nese↔English glossary (the robot's utterances are Chinese), Appendix D a file index and Appendix E the experiment log.

Three paths through the book

- **Operator** (just wants to run it): Chapter 1, then Chapter 13 § 13.6, then Appendix A.
- **Engineer** (will modify it): Chapters 3–9 in order, then Chapter 13.
- **Reviewer** (must judge it): Chapter 1, Chapter 2, Chapter 13, Chapter 14, Chapter 15.

The system at a glance

1.1 The task

A robot stands at the origin of a 17.8 × 13.8 m simulated house, in the central corridor, facing +x. Its job, in the finished system, is to accept a sentence like “带我去厨房看看冰箱” (“take me to the kitchen and look at the fridge”), decide what that means, drive there without hitting furniture, stop at a pose from which a camera can actually see the object, answer a question about what it saw, and — if the operator changes their mind halfway — abandon the trip cleanly.

Everything in this project exists to make that sentence work, and to make its failures diagnosable.

1.2 Three phases, three additions

The project grew in three phases. Later phases add capability; they never replace earlier behaviour, which is why the code has explicit switches rather than one growing pipeline.

Phase	Added	Evidence of completion
1 — classical stack	Gazebo world, robot model, TF tree, SLAM built by driving a fixed tour, Nav2 navigation on the saved map	tour map 359×278 cells at 5 cm (−0.1° tilt), Nav2 drives to arbitrary goals
2 — agent layer	semantic map, local LLM (tools + vision), natural-language control, vision confirmation, passability question answering	7 online + 6 offline acceptance items (`docs/phase2A_results.md`, `docs/phase2B_vision.md`)
3 — sensors and behaviour	64-line lidar behind a `lidar:=3d` switch, autonomous frontier exploration, mid-drive preemption, a startup assertion for the laser topic	RTF 0.98, `/scan` 8.50 Hz, exploration campaigns, 7/7 preemption criteria (`docs/phase3_lidar3d.md`)

The important structural fact: **phase 1 is frozen**. The tour script, the two-dimensional world and the maps built by the tour are byte-identical to the version that passed acceptance. Exploration, the 3D sensor and the agent layer are additions next to that baseline, never edits of it. Chapter 14 explains how that freeze is enforced (a read-only delivery snapshot).

1.3 Five layers, and the one rule that keeps them apart

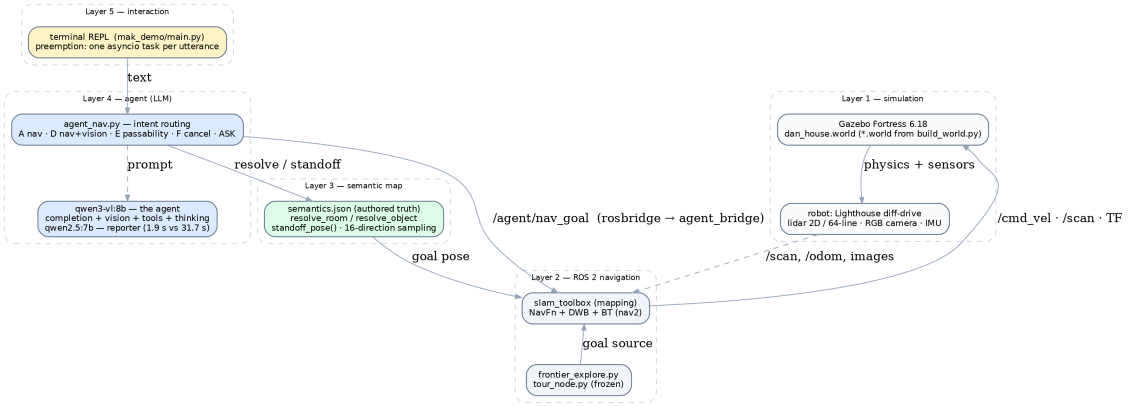


Figure 1: The five layers. Decisions flow downwards as goals; measurements flow upwards as observations. The LLM sits in layer 4 and never touches `/cmd_vel`.

The rule that makes the system debuggable is a strict separation of authority:

- **Layer 1 (simulation)** owns physics and sensors. Nothing above it is allowed to pretend a sensor is more accurate than it is.
- **Layer 2 (ROS 2 navigation)** owns motion. `Nav2` executes goals; it does not decide them.
- **Layer 3 (semantic map)** owns naming. It answers "which coordinates does the word kitchen mean", and it is a database, not a perception system (Chapter 11).
- **Layer 4 (agent)** owns intent. It may choose a goal, but a deterministic routing table decides whether the utterance is a navigation request, a question, a cancellation, or a rejection.
- **Layer 5 (interaction)** owns concurrency: one task per utterance, so a new sentence can preempt an old one without corrupting state.

A bug in this project is almost always a bug between layers: a stale goal result written into a fresh goal's state (Chapter 12), a semantic label that disagrees with the physical world (Chapter 11), a second publisher on `/scan` that silently feeds SLAM the wrong data (Chapter 6).

1.4 What the robot can do today

```
$ cd mak_demo && .venv/bin/python main.py --interactive

指令> 带我去厨房                    → drives to the kitchen
指令> 别去了，去客厅                 → cancels the kitchen leg, re-targets living room
指令> 停下                          → cancels, sends no new goal
指令> 冰箱在哪                      → answers positionally, no motion
指令> 前面能过去吗                  → laser + vision passability verdict
指令> 带我去冰箱看看                → drives to an observation pose, then VLM confirms
指令> 带我去地下室                  → refuses in 0.0 s, no motion
```

Each line is an acceptance item with a measured latency in Chapters 11–12. The refusal is not politeness: the lexicon has no mapping for basement, so the router returns before any model call (Chapter 10 § 10.5).

1.5 Acceptance at a glance

The project uses eight runtime gates plus two offline suites. The last full run of every gate (2026-10-26, isolated ROS domain 161) is the reference result:

Gate	What it proves	Result
G1 link	rosbridge, `/agent/pose`, TF all alive	24 pose frames, status idle
G1b localisation	`/odom` agrees with `map→base_link`	0.00 m / 0.0°
G2 drive	one hard-coded goal is executed	`succeeded` 13.3 s, 0.19 m error, 0 recoveries
G3 semantic	object → standoff pose → Nav2	`succeeded` 63.0 s, 0.15 m off plan
G4 vision (mode D)	drive, stop at 1.79 m, VLM names the object	observed 1.79 m, VLM 13.1 s, 4/4 checks
G5 passability (mode E)	laser and vision agree about a blocked path	laser 1.12 m, vision “blocked”, agree
G6 question (mode B)	answer without moving	41.1 s, “south, about 9.2 m”
G7 refusal	unknown place rejected, no motion	0.0 s
G8 compound	two-segment command still works	both segments `succeeded` (55.6 s + 58.2 s)
Offline suite	11 self-checks run without ROS	11/11
Scan guard	laser topic has exactly one publisher	6/6 unit cases + 4 live scenarios

Takeaways. Three phases, five layers, one authority rule. Phase 1 is frozen; phases 2–3 are additive. Eleven acceptance results — not opinions — define “working”.

2 — Eight design principles

This chapter is the constitution of the project. Every later chapter can be read as an application of one of these rules, and every rule below carries a concrete instance where it was either followed or violated and then repaired.

2.1 Simulation is a measuring instrument, not a toy

A simulator that runs at 0.4× real time silently changes every experiment: the laser publishes at 4 Hz instead of 10, the controller's progress checker times out, and you spend a day debugging a "navigation bug" that is really a frame-rate bug.

Therefore the project treats real-time factor (RTF) as a first-class quantity and provides four levers: run headless (no GUI), keep the physics step at 1 ms, avoid textures you do not need, and offload rendering to the GPU when the GUI is needed. Measured RTF in the finished system: **0.99** (2D sensors) and **0.98** (64-line lidar). Earlier, a GUI session with a heavy world produced `/scan` at 5 Hz and the whole stack looked broken.

2.2 One source of truth for the world

`worlds/*.world` and `config/semantics.json` and `config/route.json` are **generated**. Their single source is `build_world.py` (898 lines). Hand-editing a `.world` file is forbidden because the next regeneration silently discards the edit — and because three consumers (the physics world, the SLAM ground truth of what exists, and the tour route) would then drift apart without anyone noticing.

```
build_world.py  ──generates──▶ worlds/dan_house.world
                ──generates──▶ worlds/dan_house_3d.world
                ──generates──▶ config/semantics.json      (7 rooms + 61 objects)
                ──generates──▶ config/route.json          (32 tour waypoints)
```

The payoff appears in Chapter 11: because the semantic map is generated from the same table that places the furniture, object coordinates are exact to the millimetre — a deliberate choice with a matching cost, also documented there.

2.3 The model decides what, code decides whether and how

The single most important structural decision of phase 2. An early version let the LLM decide everything, including whether a request was safe to execute. It produced plausible navigation to places that did not exist and, worse, could not be debugged: the failure was inside a sampled token.

The final design moves authority down:

Question	Decided by
Is this a navigation request, a question, a cancellation or a refusal?	a deterministic table (`preempt.py`, `planner.py`)
Which room or object does the phrase mean?	lexicon + scoring (`semantic_map.resolve_*`)
Where should the robot stand to see the object?	geometry, 16-direction sampling (`standoff_pose`)
What is the overall plan for a multi-clause sentence?	LLM (`qwen3-vl:8b`) with tools — but code decides the route
Is the camera actually looking at a fridge?	VLM (`qwen3-vl:8b`) with closed options

The LLM never emits a velocity. It emits intent plus arguments; code validates the arguments against the semantic map before anything moves.

2.4 One 2D contract in the middle

Every consumer above layer 2 — SLAM, both costmaps, the exploration node, the agent's passability check — reads the same message: `sensor_msgs/LaserScan` on `/scan`, 360 rays, 0.12–12.0 m, `frame_id=base_scan`.

That is why the 64-line upgrade did not ripple through the system: the 23,040-point cloud is projected into a single 2D slice (`pointcloud_to_laserscan`, −0.75 m … +0.05 m) and everything downstream is unchanged. The cost of this contract is stated plainly in Chapter 14: the costmap cannot see what sits above the slice.

2.5 Freeze what passed; add next to it

Acceptance artefacts are immobilised rather than trusted to discipline:

- `tour_node.py` and the 2D world are untouched since their acceptance run;
- historical maps live under `config/maps/preserved/`;
- the five deliverable documents are snapshotted read-only with a SHA-256 manifest (`delivery/2026-10-26/`);
- exploration experiments may only modify `config/nav2_params_explore*.yaml` and `config/nav_to_pose_explore.xml`, never the acceptance parameters.

2.6 Measure with one decoder, and never trust a single map statistic

Map quality is computed by exactly one module, `tools/map_quality.py`, used by every probe and by the experiment driver. The reason is concrete: a second decoder that treated "unknown" as "free" reported exploration finished while 180 m^2 of the house had never been seen. One decoder, one definition of free/occupied/unknown, no contradiction between tools.

2.7 One run per arm is not an experiment

The project learned this the expensive way. A single 40-goal run said planner A was better; three runs per arm said the same thing more strongly but also revealed that the spread inside an arm exceeded every difference between arms for the clearance parameter. The rule that followed: for any A/B claim, $\geqslant 3$ repetitions, $\geqslant 40$ goals, compare medians, and report the within-arm range next to the between-arm difference. Chapter 9 § 9.6 shows both the successes and the retracted claim.

2.8 Fail loudly at startup, report honestly at the end

Two mechanisms embody this rule.

Startup. `tools/scan_pub_guard.py` asserts that `/scan` has exactly one publisher and that the publisher is the expected node. A second publisher is never transient (it means a bridge line was added twice), so the guard fails immediately; zero publishers is tolerated only inside a startup grace window.

Shutdown. A cancelled navigation leg must never report arrival. This was a real defect: after a preemption, the cancelled leg printed `failed` and, worse, an unrelated stale result could mark a new goal as failed. The fix (per-goal sequence numbers, Chapter 12) is now enforced by a grep-level assertion inside the offline test suite, so it cannot silently regress.

> **Takeaways.** Instrument the simulator; generate the world from one source; keep authority for safety in code and intent in the model; one 2D contract; freeze accepted artefacts; one decoder; three runs where two would look sufficient; make failures loud and lies impossible.

3 — The house: world generation

3.1 Why the world is code

The simulated house is not a hand-drawn `.world` file. It is emitted by `build_world.py` (898 lines) from a small table of rooms, a corridor width and a set of furniture placements. Three reasons, all of which cost real debugging time before the rule was adopted:

1. **Consistency.** The tour route, the semantic map and the physics geometry must agree. Generating all three from one table makes disagreement impossible.
2. **Regeneration.** Changing a door width means editing one constant, not hunting for occurrences in XML. The finished world has 136 `<model>` elements; no human should maintain them by hand.
3. **Diffability.** A generated file is 165 lines / 676 KB of XML, and it never needs a code review. The table is what gets reviewed.

   $ python3 build_world.py # 2D world → worlds/dan_house.world $ python3 build_world.py –lidar 3d # 3D world → worlds/dan_house_3d.world

3.2 Geometry in one table

```
XW, XE = -9.0, 9.0      # outer wall centre lines  → 18 m
YS, YN = -7.0, 7.0      # outer wall centre lines  → 14 m
COR    = 1.5            # corridor walls at y = ±1.5  → 2.88 m clear
DOOR_W  = 1.3           # standard door width
DOOR_W_BATH = 1.2       # bathroom door

ROOMS = [               # name     x0      x1      y0      y1    floor material
    ('living',  -8.89, -3.20,  1.56,  6.89, WOOD),
    ('kitchen', -3.08,  1.60,  1.56,  6.89, TILE),
    ('study',    1.72,  8.89,  1.56,  6.89, WOODD),
    ('bed1',    -8.89, -4.00, -6.89, -1.56, CARP),
    ('bed2',    -3.88,  0.90, -6.89, -1.56, CARP),
    ('bath',     1.02,  3.60, -6.89, -1.56, TILE),
    ('store',    3.72,  8.89, -6.89, -1.56, WOODD),
]
```

The floor therefore spans 17.8 × 13.8 m of room area; older notes round it to “16 × 12 m”, which is a historical approximation, not a measurement. Door centres sit on the corridor walls at x = −6.0 (living), −0.8 (kitchen), +5.2 (study), −6.3 (bed 1), −1.5 (bed 2), +2.3 (bath), +6.2 (storage); kitchen and study are additionally joined by an internal door at x = 1.66, y = 3.8.

Why these dimensions? They are a compromise between three constraints that all bite at once: the

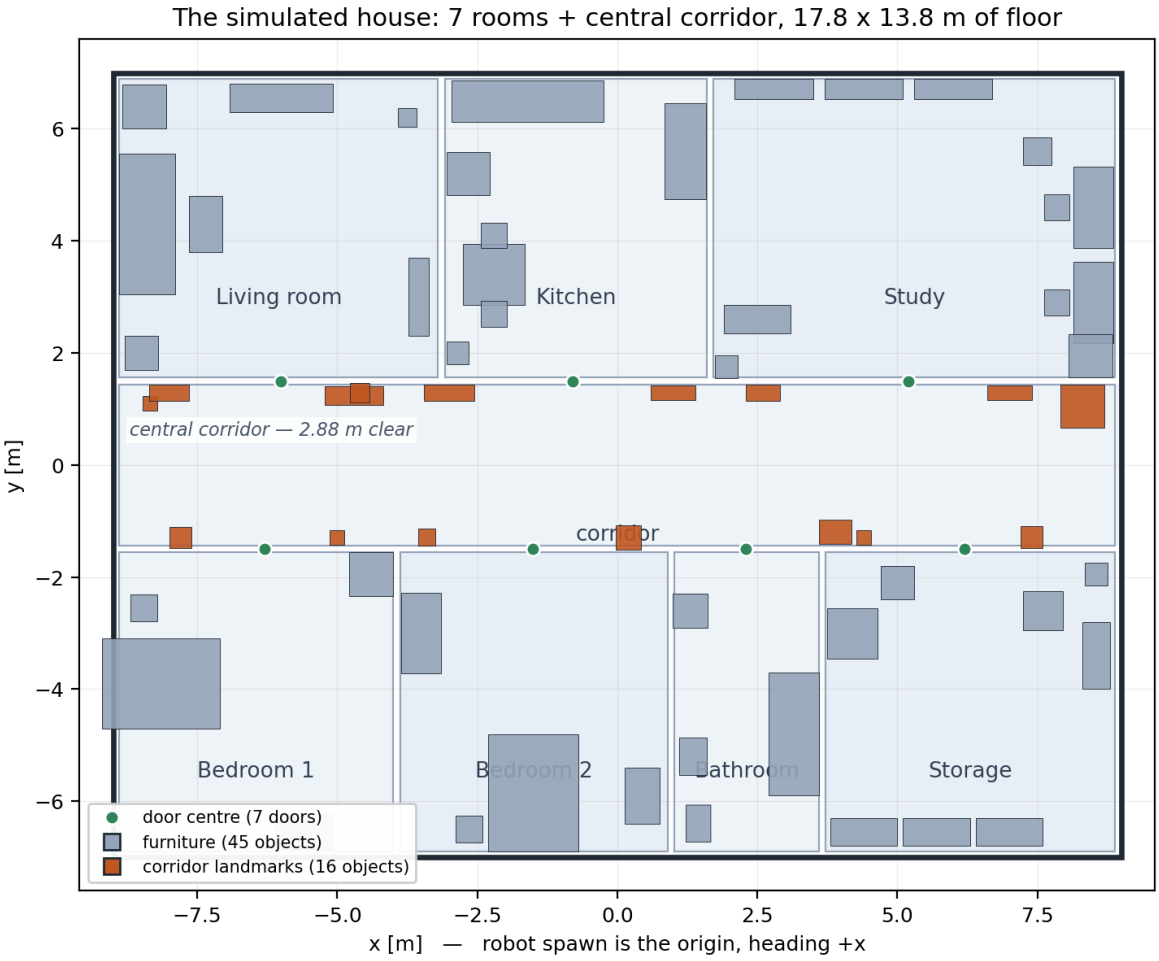


Figure 2: The generated house. Grey boxes are furniture; orange boxes are the sixteen corridor landmarks discussed in § 3.4.

robot is 0.42 m wide, so the 2.88 m corridor leaves 2.23 m of slack; a 1.3 m door lets the 0.46 m costmap footprint through with room to spare; and a 5 cm map resolution at this size gives a 359 × 278 cell map — small enough that SLAM stays fast, large enough that five rooms are more than a corner.

3.3 Furniture, and the two-height trick

61 objects in 33 classes are placed by the same script. Each placement calls one helper:

```
def foot(cx, cy, w, d, rz, name, cls=None):
    if abs(abs(rz) - math.pi / 2) < 0.2:   # at ±90° swap width and depth
        w, d = d, w
    FOOT.append((cx, cy, w/2.0, d/2.0, name))
    SEM.append({'id': ..., 'class': cls or name, 'room': room_of(cx, cy),
                'x': cx, 'y': cy, 'yaw': rz, 'w': w, 'd': d})
```

That three-line swap has bitten every downstream consumer at least once: because `w`/`d` are already exchanged for ±90° poses, a visualiser that also rotates the box by `yaw` draws it rotated twice and appears to push furniture through the wall. Chapter 14's figure-generation script documents the correct rule.

The two world variants differ in one respect:

	2D world (`dan_house.world`)	3D world (`dan_house_3d.world`)
Lidar	one scan line at 0.78 m	64 lines, −20° … +15°
Low furniture	**raised** so its top is ⩾ 0.85 m (`RAISE_LOW`)	real dimensions
Purpose	honest SLAM baseline for a single-line sensor	see what a real depth sensor sees

Raising furniture in the 2D world is not cheating: a single-line lidar genuinely cannot see a 0.4 m coffee table, and if the map is expected to look like the house, the geometry has to be visible to the sensor. The 3D variant is where the real heights live — and where the costmap's blindness to overhanging geometry becomes visible (Chapter 14).

3.4 The corridor landmarks, and why the label does not match the position

Sixteen of the 61 objects have `|y| < 1.5`, i.e. they physically stand in the corridor: consoles, a cart, stools, umbrella stands, a fire extinguisher. `room_of()` applies a 0.3 m tolerance, so each of them receives a room label — `shoe_cabinet_1` is labelled `living_room` while standing at y = 1.29 in the corridor.

This is **intentional**: the semantic layer records ownership ("this shoe cabinet belongs to the living room"), not a polygon test. It also produces eight "not in room" warnings during semantic self-checks, which are correct warnings about a wrong assumption, not bugs. Removing them by editing the JSON would delete a deliberate design decision in order to silence a diagnostic.

3.5 Self-checks that run with the generator

Generation validates itself before anything is ever launched:

```
① route geometry   min clearance between 32 waypoints: 42 cm PASS
                  7 door passages traversable PASS
② SDF validity     gz sdf -k worlds/dan_house_3d.world → Valid PASS
③ see-ability      12/12 landmarks detected from the tour poses PASS
                  potted_plant south wall: theory 1.09 m, measured 1.10 m
                  shoe_cabinet north wall: theory 2.60 m, measured 2.65 m
```

Step ③ is the interesting one: it closes the loop between "the world says there is a plant there" and "the laser can actually return a point for it". A landmark that exists but is invisible to the sensor is indistinguishable from a landmark that does not exist, and the semantic layer would be quietly wrong.

Takeaways. The world is generated, never edited. One table drives physics, semantics and the tour route. Two world variants exist because a single-line lidar and a 64-line lidar do not see the same house. Object labels record ownership, not physical containment.

4 — The robot: Lighthouse

4.1 One robot, two sensor heads

Lighthouse is a two-wheel differential drive platform with two caster wheels, a sensor tower, and a single RGB camera. It exists in two variants that share the chassis but differ in the lidar:

	`models/lighthouse/`	`models/lighthouse_3d/`
Lidar	single-line `LaserScan`, topic `/scan`	64-line `gpu_lidar`, topic `/scan/points` via bridge
Vertical field	one plane at z ≈ 0.78 m	−0.349066 … +0.261799 rad (−20° … +15°)
Points per frame	360	360 × 64 = 23,040 (≈ 720 KB, ≈ 7 MB/s at 10 Hz)
Status	**frozen** (phase 1 baseline)	active for phases 2–3 experiments

The 3D head is mounted on a mast whose top sits at 0.833 m; the scan joint is at 0.770 m. The camera sits lower, at 0.70 m, on purpose — that height is an empirical choice explained in Chapter 11 § 11.4.

4.2 Chassis numbers that the rest of the stack depends on

```
body            0.306 × 0.281 m       mass 1.3729096 kg
wheel radius    0.033 m             wheel separation 0.288 m
costmap radius  0.23 m              (circumscribed footprint ≈ 0.21 m + margin)
height          0.85 m (tower top)  camera 0.70 m   lidar (3D) 0.833 m
```

Three of these are load-bearing for later chapters: `robot_radius = 0.23` decides whether the robot fits through a 1.3 m door with a 0.45 m inflation radius (it does, with 0.02 m to spare per side); `0.288 m` is what the wheel odometry uses; and the 0.033 m wheel radius is the scale factor that made the "wheels sinking into the floor" bug so confusing (§ 4.4).

4.3 The TF tree is a contract

```
# dan_house.launch.py — the two switches that change this tree
'loc' : 'amcl'  | 'truth'    # who owns map -> odom
'ekf' : 'false' | 'true'     # who owns odom -> base_footprint
```

- `map→odom` is published by `slam_toolbox` in mapping mode, or by a static identity transform when `loc:=truth` (the simulator's `/odom` is ground truth), or by AMCL in the real-robot configuration.

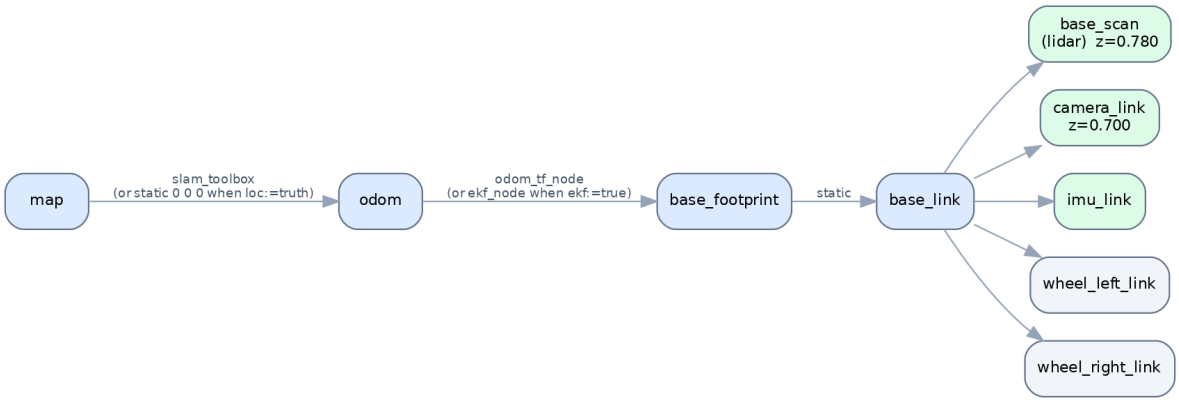


Figure 3: TF tree. Everything above `odom` is a decision (SLAM, or a static zero when truth poses are used); everything below `base_footprint` is mechanical.

- `odom`→`base_footprint` is published by `odom_tf_node.py` (a 40-line node that integrates the ground-truth pose) or by `robot_localization`'s `ekf_node` when `ekf:=true`.
- Everything below `base_link` is static: `base_scan`, `camera_link`, `imu_link`, the two wheels, the casters.

The rule that follows: **no consumer may assume a transform is fresh**. `slam_toolbox` publishes `map`→`odom` at 50 Hz (`transform_publish_period: 0.02`) precisely because a stale `map`→`base_link` makes the controller plan from a pose the robot no longer occupies.

4.4 Four hardware bugs worth remembering

(a) Wheels sinking into the floor. The chassis was placed with its origin at the axle height, but the wheel collision geometry extended below the ground plane; the physics engine then pushed the whole body up by a few millimetres every step. Symptom: odometry drifted even when standing still. Fix: correct the link origins so the wheel's lowest point is exactly at z = 0 in the spawn pose.

(b) The robot lifting its nose. Asymmetric caster placement meant the front casters carried more load; during hard acceleration the body pitched. Measured `pitch ≈ 0.0004 rad` at rest and a visible lift under acceleration. Fix: symmetric caster geometry plus inertia values matching the real mass distribution.

(c) `/scan` at 5 Hz. Not a sensor bug at all — RTF was 0.5 because the GUI was rendering a textured world. Fix: headless server plus the RTF levers of Chapter 2 § 2.1.

(d) The lidar seeing its own tower. The mast stood 0.089 m in front of the scan plane, and the laser returned 10 echoes at 0.124–0.138 m, angles −24° … −31°, which in the body frame is `(0.054, −0.055)` — i.e. the mast, not the chassis. Because the passability threshold was `MIN_R = 0.12`, those echoes passed the filter, and an open corridor was reported as "blocked right at the nose". Fix: move the sensor out of the housing, then add a minimum-range guard that is smaller than the true obstacle distance rather than tuned to the artefact.

Lesson: two of these four looked like software bugs for hours. A simulator will happily let you debug a geometry error as if it were a control error.

4.5 Sensors the agent layer can use

Sensor	Rate	What it is good for	What it must not be used for
`/scan` (LaserScan)	8.4–8.5 Hz measured	SLAM, costmaps, passability, frontier detection	seeing above the slice height
`/scan/points` (3D)	8.4 Hz measured	future 3D costmap, offline analysis	direct consumer today (nothing subscribes)
`/camera/image_raw`	7.1 Hz, 320×240 rgb8, HFOV 1.204 rad	VLM confirmation, passability second opinion	geometry, scale, depth (monocular, no depth topic)
`/imu`	100 Hz	EKF experiments (mode C)	accepted navigation (unused there)
`/odom`	50 Hz	truth-pose navigation, TF	real-robot localisation (that is AMCL's job)

The camera row matters for two later chapters: the camera has **no depth channel at all** (Chapter 11), and it is deliberately not part of the mapping pipeline (Chapter 3 of the project's original design notes) — it is a semantic instrument, not a geometric one.

Takeaways. Two sensor heads, one chassis. The TF tree encodes who owns which decision. Two of the four hardware bugs presented as software bugs. The camera has no depth, which determines what the agent layer can and cannot do with it.

5 — The runtime: one launch file, fifteen switches

5.1 Why one launch file

`dan_house.launch.py` (495 lines) starts everything: the simulator, the bridge, the scan projection, the TF nodes, SLAM, Nav2, exploration, the tour and RViz. A single entry point is a deliberate trade: the file is long, but there is exactly one place where the wiring order lives, and the wiring order is the part that breaks (Chapter 9 records two separate incidents caused by a lifecycle bring-up race).

Everything else is a switch. Nothing is deleted when a subsystem is superseded; it is gated.

Argument	Default	Meaning
`lidar`	`2d`	`2d` → single-line model + `/scan` bridged directly; `3d` → 64-line model + projection
`slam`	`false`	start `slam_toolbox` (mapping mode)
`nav`	`false`	start Nav2 (`nav2_bringup`, or the truth-pose variant)
`loc`	`amcl`	`amcl` → AMCL owns `map→odom`; `truth` → static identity, `/odom` is trusted
`ekf`	`false`	`odom→base_footprint` from `robot_localization` instead of `odom_tf_node.py`
`tour`	`false`	run the frozen 32-waypoint tour (`tour_speed`, `tour_spin_rate`)
`explore`	`false`	run `frontier_explore.py` (see its own six parameters below)
`rviz`, `headless`, `nvidia`	`false`	display, no-GUI server mode, GPU offload
`map`	(2D map)	map YAML to load when navigating
`use_sim_time`	`true`	almost never changed

Exploration owns six more parameters, all of which are measurement devices rather than preferences: `explore_max_goals` (120), `explore_max_minutes` (60), `explore_min_cluster` (12), `explore_revisit_gap` (0.4), `explore_goal_clearance` (0.40), `explore_min_goal_dist` (0.60), `explore_stall_secs` (12.0).

5.2 The order matters: a timetable, not a list

Subsystems are not started simultaneously. Each delay exists because of an observed failure:

```
t = 0 s    gz server (and gz GUI unless headless)
t = 2 s    robot_state_publisher        (needs the URDF serviced by the world)
t = 3 s    pointcloud_to_laserscan      (harmless early; it only works when subscribed)
t = 4 s    slam_toolbox
t = 4 s    static map→odom + map_server + lifecycle (only when loc:=truth)
t = 6 s    Nav2 bringup                 (after SLAM has published its first map)
t = 10 s   tour_node
t = 20 s   nav_live (exploration's Navigator client — needs Nav2 action servers)
t = 30 s   frontier_explore
```

The 20 s gap before `nav_live` is not superstition: Nav2's `lifecycle_manager` can sit in Configuring for many seconds on a loaded machine, and an exploration node that calls `NavigateToPose` before the action server exists fails instantly and (in an early version) retried forever against a dead server. The same race produced a 25-minute run that issued zero goals; Chapter 9 § 9.5 shows the guard that now turns that failure into a loud error instead of an empty map.

5.3 The bridge contract

`ros_gz_bridge` bridges ten topics. In 3D mode exactly one of them changes:

Topic	Type	2D	3D
/clock	rosgraph_msgs/Clock	√	√
/odom	nav_msgs/Odometry	√	√
/odom_wheels	nav_msgs/Odometry	√	√
/imu	sensor_msgs/Imu	√	√
/joint_states	sensor_msgs/JointState	√	√
/cmd_vel	geometry_msgs/Twist	√	√
/scan	sensor_msgs/LaserScan	√ (direct)	**x never**
/scan/points	sensor_msgs/PointCloud2	x	√
/camera/image_raw	sensor_msgs/Image	√	√
/camera/camera_info	sensor_msgs/CameraInfo	√	√

The 3D row marked "never" is a rule with a mechanism behind it (Chapter 6 § 6.5) — the gz 64-line sensor emits both a LaserScan and a point cloud, and bridging the wrong one would feed SLAM a second, silently conflicting source.

5.4 Reality check before every launch

`preflight.sh` is the gate between "I think nothing is running" and "I am about to start a second physics engine". It checks three things and refuses to be optimistic:

```
[1] count gz processes, print age + partition + domain of each
    → an "orphan gz" (server alive, bridge dead) burns CPU and VRAM while publishing nothing
[2] publisher count of /odom, /scan, /clock must each be exactly 1
    /cmd_vel publisher count is printed (>4 usually means a leftover tour or teleop node)
[3] no duplicate node names (two full stacks look exactly like one broken stack)
```

Its most useful output is the distinction between just starting (gz younger than 45 s, no data yet — wait 20–30 s and re-run) and orphaned (gz older than 45 s with no data — the bridge died and the physics engine is idling).

5.5 Isolation: how experiments avoid touching the live map

Every experiment driver run allocates a fresh DDS domain and a fresh Gazebo partition:

`ROS_DOMAIN_ID=120..129, 130, 140, 150/151, 160, 161 ...` `IGN_PARTITION=ab120, ab160, ...`

Two consequences, both required for trustworthy measurements: (1) a run cannot accidentally subscribe to the live stack or overwrite the live map, and (2) two runs can be inspected side-by-side afterwards, because their logs and map prefixes are distinct (`..._ab_1025_2047_0.40_1_checkpoint_48`). The rule is enforced socially — it is the first thing the driver does, and every acceptance log prints its domain so ambiguity is impossible after the fact.

> **Takeaways.** One launch file, all behaviour gated by switches. Timers encode observed failure modes, not taste. The bridge table is a contract with one forbidden cell. Preflight distinguishes "loading" from "orphaned". Isolation is the first line of defence for every measurement.

6 — The laser contract: from photons to /scan

6.1 Two sensors, one message

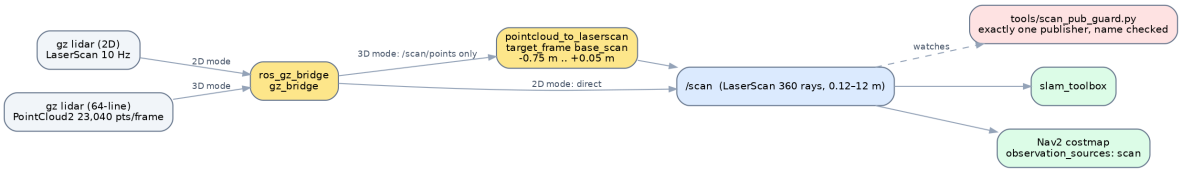


Figure 4: The scan pipeline in both modes. Note the forbidden edge: in 3D mode the gz LaserScan is never bridged.

The entire stack above Nav2 consumes one message type. Everything in this chapter exists to guarantee that `/scan` means exactly one thing.

2D mode. The gz single-line lidar publishes `LaserScan` as topic `scan`; the bridge maps it straight to ROS `/scan`. Nothing else touches the topic.

3D mode. The gz 64-line lidar publishes `PointCloudPacked` on `scan` and — this is the trap — also a derived `LaserScan`. The bridge is configured to accept **only** the point cloud, renamed `/scan/points`. A separate node projects 3D into 2D:

```
pointcloud_to_laserscan_node:
  ros__parameters:
    target_frame: base_scan
    min_height: -0.75       # below the caster contact patch
    max_height: +0.05       # a horizontal slice just above the floor
    angle_min: -3.14159
    angle_max: +3.14159
    angle_increment: 0.0174533    # 360 rays
    range_min: 0.12
    range_max: 12.0
    use_inf: true
```

`range_min`/`range_max` are deliberately identical to `slam_toolbox`'s `min_laser_range`/`max_laser_range` (0.12 / 12.0) and to the sensor's own limits: a mismatch here produces a map that is subtly scaled, and scaled maps are the hardest maps to debug because they look fine.

Measured end-to-end in 3D mode: `/scan` at **8.50 Hz**, `/scan/points` at **8.40 Hz**, **359 of 360** rays returning an echo at the spawn pose, RTF **0.98**, and exactly **one** publisher on `/scan` (`/pointcloud_to_laserscan`). The corresponding 2D numbers are 0.99 RTF and 8.5+ Hz.

6.2 Why a 2D slice, and what it costs

Projecting 23,040 points per frame to 360 rays throws away 98.4 % of the information. That is the price of a contract: every consumer above works in 2D, the costmaps are 2D, and `slam_toolbox` is a 2D graph SLAM.

What the slice cannot see:

- anything entirely above +0.05 m and below the next structure (e.g. a table top, a shelf, an overhanging pallet);
- anything below −0.75 m (nothing, by construction);
- therefore the costmap is blind to overhangs, and a robot could in principle drive under an object it cannot pass beneath in 3D.

This is not a bug, it is a documented boundary of the design, and it is the reason a warehouse world would force an architectural change (Chapter 15 § 15.3). The 3D cloud is already being published and bridged; nothing consumes it yet.

6.3 The failure that motivated an assertion

During phase 3, a launch edit briefly bridged the gz LaserScan in 3D mode as well. The result was not an error message. It was:

- SLAM receiving two `/scan` streams whose timestamps and ray counts differed slightly;
- the map ghosting (double walls) in the corridors;
- Nav2 planning through geometry that one of the streams did not see;
- an afternoon of hypothesis-testing the wrong subsystem.

The failure mode is uniquely nasty because DDS happily delivers both, and which one a subscriber uses depends on arrival order. Nothing in ROS warns about it.

6.4 The guard: one publisher, and it must be the right one

`tools/scan_pub_guard.py` runs as a short-lived node inside the launch (resident with `--watch`), and judges the topic, not the count alone:

Observation	Verdict	Why
0 publishers	fail	projector or bridge never started; but silent during the startup grace window
1 publisher, node name matches `--expect`	pass	the normal case
1 publisher, wrong node name	fail	right count, wrong source — e.g. the bridge when the projector was expected
$\geqslant 2$ publishers	fail **immediately**	never transient; always a wiring error

The three design choices worth keeping if you port this elsewhere:

1. **Check identity, not just count.** A count-only guard passes the most likely real mistake (wrong bridge line), because it still yields exactly one publisher.

2. **Never delay the ⩾2 verdict.** Startup grace exists for "not yet", never for "impossible".
3. **Stay resident.** With `--watch` the guard also catches a rogue publisher started later from another terminal, and it recovers by itself when that publisher exits, so nobody has to relaunch the stack.

Evidence from the four live scenarios (Chapter 13 § 13.5 reproduces the commands):

```
3D normal            PASS /scan publisher 1: /pointcloud_to_laserscan
3D negative          FAIL /scan has 2 publishers: /_ros2cli_12764, /pointcloud_to_laserscan
                       (a second publisher was injected on purpose; caught within 30 s)
rogue publisher gone PASS recovered without relaunch
2D normal            PASS /scan publisher 1: /gz_bridge
```

Plus six offline unit cases for the judging function, so the logic can be changed without a simulator (0 publishers, correct single, wrong single, mixed pair, namespaced node, and the 2D expectation).

> **Takeaways.** One message type is the spine of the stack. The 3D sensor is degraded to a 2D slice by contract, and that boundary is documented rather than denied. Duplicate publishers fail silently by nature, so the project asserts both how many and which one.

7 — SLAM: building the map that everything else trusts

7.1 Configuration

Mapping is `slam_toolbox` in asynchronous mapping mode. The parameter file is short and every line has a reason:

```yaml
slam_toolbox:
  odom_frame: odom
  scan_topic: /scan
  mode: mapping                  # mapping | localization
  resolution: 0.05               # 5 cm — fine enough for a 1.3 m door, cheap enough to be fast
  map_update_interval: 2.0       # 5.0 → 2.0: the live map is also exploration's planning input
  min_laser_range: 0.12          # identical to the sensor and to the projection
  max_laser_range: 12.0          # identical to the sensor and to the projection
  transform_publish_period: 0.02 # 50 Hz: Nav2 must never plan from a stale map→odom
  minimum_travel_distance: 0.3   # keep the default: smaller values make the solver unstable
  minimum_travel_heading: 0.3
  use_scan_matching: true
```

Two of these are not defaults and both were chosen after measurement: `map_update_interval: 2.0` because exploration plans against the live map (a 5 s-stale map means the frontier list is stale too), and `transform_publish_period: 0.02` because at 10 Hz the controller occasionally read a transform 100 ms old and planned from the wrong pose.

7.2 What the map looks like, quantitatively

Metric	Value	How it is measured
Size	359 × 278 cells	PGM header
Extent	17.95 × 13.90 m	cells × 0.05 m
Tilt	−0.1°	Hough-style fit in `tools/map_quality.py`
Interior unknown	**13.5 m²**	unknown cells inside x∈[−7.9,7.9], y∈[−5.9,5.9] × 0.0025
Live 3D map	`config/maps/dan_house_lighthouse3d.yaml` → `..._explore4.pgm`	

The “interior unknown” definition matters and is used everywhere in this book: it counts unknown cells **only inside the house footprint**, so the permanently-unreachable ring outside the walls is not counted as coverage loss. Without that restriction every run reports a similar large number and improvements become invisible.

7.3 The ghosting investigation, and one retracted conclusion

Early maps had double walls in the corridors and a visible rotation between the north and south halves. The investigation proceeded in the only order that works:

```
① is /odom trustworthy?      compare /odom against the simulator's ground-truth pose
② how big is SLAM's patch?   print map→odom translation and rotation over time
③ score the map objectively  tilt, extent, unknown area — not "it looks bad in RViz"
```

Step ① found `/odom` matching ground truth to below 1 mm. Step ② found `map→odom` carrying a 1.25 m / 124.5° “correction” in one session — which led to a diagnosis of an AMCL defect, a switch to truth poses for simulation, and a paragraph in three documents.

That conclusion was later withdrawn. The jump was caused by a manual misconfiguration of the simulator during that session, not by AMCL or by the stack. The method survives (compare `/odom` with `map→base_link` before reading Nav2 logs — Nav2 will dutifully report “剩余 7.2 m” for a 4.6 m trip and look like a planner bug), and `loc:=truth` remains the recommended simulation setting, because in simulation `/odom` is truth and AMCL can only add error.

The documents record this as a correction with a date rather than an edit, because a future reader who finds the old numbers must be able to see that they were measured, and why they were misinterpreted.

7.4 Localisation: three modes, one unfinished branch

Mode	`map→odom` source	Used for	Status
`loc:=truth`	static identity transform	all acceptance runs, all exploration experiments	**verified**
`loc:=amcl`	AMCL on the saved map	the real-robot configuration	**not re-verified** since the truth shortcut was adopted
`ekf:=true`	`robot_localization` fusing wheels + IMU	“closer to reality” experiments	measured drift 7.1 m / −37° over a long run — unsolved

That middle row is the honest gap in this project (Chapter 14 § 14.4). Everything you can measure in this book was measured with truth poses. A real robot has no truth pose, so the first task on hardware is not the agent layer — it is making `loc:=amcl` hold `map→base_link` within 0.3 m over a long drive.

7.5 Map hygiene: checkpoints, preservation, naming

A map is evidence, so it is never overwritten by an experiment.

```
config/maps/
  dan_house_lighthouse3d.*                    live map for phase 2/3 acceptance
```

```
dan_house_lighthouse.*                        2D baseline (frozen)
..._explore_checkpoint_08/16/24/32.*          tour checkpoints (every 8 waypoints)
..._ab_1025_1647_*_checkpoint_NN.*            3×3 clearance campaign, 10 runs
..._ab_1025_2047_0.40_{1,2}_checkpoint_NN.*   80-goal coverage campaign
preserved/                                    four historical maps, untouched
```

Naming carries the experiment tag, the arm and the repetition, because the alternative — one file named `map` overwritten by whichever run finished last — is how an experiment loses its evidence. The same rule appears in the driver (Chapter 9 §9.5): a retry writes to a new domain-suffixed filename instead of overwriting the failed attempt.

7.6 Known SLAM weaknesses

- **Radial spray.** Rays that never return are still painted as free space at their far end, so the map has thin free streaks in unexplored directions. It affects map aesthetics and marginally the frontier detector; it does not affect navigation.
- **Long-haul drift is unmeasured.** The house allows a loop of ~60 m; a warehouse aisle loop is several hundred metres. Nothing in this project has measured loop-closure quality beyond the house.
- **AMCL on a ghosted map is unreliable** — which is why phase 2 uses truth poses, and why the real-robot path must begin with a clean map and a localisation acceptance test.

Takeaways. SLAM parameters are chosen to serve exploration's freshness needs. Map quality is a number, computed by one decoder. One published conclusion in this project was later withdrawn — and the withdrawal is part of the record. The localisation branch that a real robot needs is the branch that is not verified here.

8 — Nav2: making a 0.42 m robot fit a 1.3 m door

8.1 Four parameter files, four jobs

The project keeps navigation parameters physically separated by role, not by convenience:

File	Used by	Velocity	Purpose
`config/nav2_params.yaml`	mapping runs	`max_vel_x 0.15`	slow, safe mapping
`config/nav2_params_nav.yaml`	acceptance / agent runs	`max_vel_x 0.26`, `min_vel_x −0.15`	faster; the file behind every number in this book
`config/nav2_params_explore.yaml`	exploration	as above	exploration-specific planner and BT
`config/nav2_params_explore_perm.yaml`	permanent arm	—	control arm copy (identical except comments)

The rule: **experiments may only modify the `*_explore*` files and `nav_to_pose_explore.xml`.** The two acceptance files are frozen. This is what makes "the acceptance still passes" a meaningful statement three phases later.

8.2 The numbers that decide feasibility

```
robot_radius            0.23 m     (circumscribed footprint ≈ 0.21 m + margin)
inflation_radius        0.45 m
cost_scaling_factor     3.0
door width              1.30 m
corridor clear width    2.88 m
goal tolerance (xy)     0.20 m
progress checker        0.25 m within 6.0 s   (exploration: 0.25 / 6.0)
```

Two consequences worth internalising before touching anything:

(a) The door is not the problem. A dwell-time-free geometric check of every door's widest path gave half-widths of 0.50–0.55 m, all ⩾ `inflation_radius` 0.45, i.e. cost **0** at the narrowest point. "The robot cannot get into the study because the door is too narrow" was falsifiable in 20 minutes of computation, and false.

(b) The goal point can be the problem. The same probe applied to failed goals found their own clearance to be 0.35–0.40 m — just inside the inflation radius. Inside that band the cost field is non-zero (132–177 units per cell in the observed cases), and DWB must traverse the entire band to reach the goal. It frequently prefers to rotate in place instead. That is the mechanism behind most "the robot reaches the door and then spins forever" failures, and it is a goal placement bug, not a planner bug.

8.3 Where the costmap comes from

```
local_costmap:
  plugins: ["voxel_layer", "inflation_layer"]
  voxel_layer:
    observation_sources: scan
    scan: { topic: /scan, data_type: "LaserScan", max_obstacle_height: 2.0,
            raytrace_max_range: 5.0, obstacle_max_range: 4.0 }
```

Note what is not there: no point cloud source, no camera. Four parameter files, all with `observation_sources: scan` and `data_type: "LaserScan"`. The `VoxelLayer` is used as a 3D container for a 2D input — it does not make the obstacle layer three-dimensional.

Consequence: everything the projection slice cannot see is invisible to navigation, including furniture overhangs, table tops and anything above 0.85 m in the 2D world because it was raised there deliberately.

8.4 Why the controller "decides" to stand still

The single most useful debugging artefact in this project is the trajectory evaluation dump. In a stall, all 419 candidate trajectories were valid — and the chosen one had `vx = 0.000` with a score of 63.40, beating a `vx = 0.103` trajectory that scored 72.70. The slower trajectory won because the faster one had to cross `BaseObstacle` cells with `raw = 195` (i.e. ~0.30 m from an obstacle).

So the robot was not stuck, confused, or blind. It was **optimising correctly against a cost function that made standing still rational**. This is why the project's first question about any stall is "which critic dominates?" and not "is the controller broken?".

Two corollaries:

- "No valid trajectories = 0" is not a health metric. A healthy-looking zero coexists with a completely motionless robot.
- Tuning the stall timer treats nothing. It was tried: an 18 s watchdog produced no improvement (4 vs 2 stalls, worse coverage 56.5 vs 45.0 m^2), so the 12 s setting was kept and the effort moved to goal reachability.

8.5 Behaviour tree

Two trees exist. The acceptance tree is stock `navigate_to_pose_w_replanning_and_recovery`. The exploration tree (`config/nav_to_pose_explore.xml`, 48 lines) changes the recovery structure — notably a BackUp-first branch, which was A/B tested: it survives more stalls (stalls 2 vs 4) but is 44 % slower and covers less area (37.6 vs 45.0 m^2). It was not adopted; the measurement is kept because the question ("would recovery behaviour help?") will be asked again.

8.6 Progress checker, and what it really measures

A 0.25 m / 6 s threshold sounds generous; against a 0.15 m/s mapping speed it is ~1.5 m of expected travel. In practice the checker fires during legitimate slow manoeuvres (turning in a doorway at 0.1 m/s with 0.3 m/s^2 acceleration). The lesson recorded in the project notes: the progress checker detects "no displacement", not "no progress"; a robot rotating to align with a door is making progress while failing the checker. Exploration therefore treats a progress-checker stop as a candidate stall, filtered by the observed rotation rate.

> **Takeaways.** Four parameter files, one freeze rule. Doors are innocent; goal clearance is guilty. The costmap sees exactly one horizontal slice. The controller stalls because standing still is cheaper — read the critics, not the symptoms. Recovery experiments are measured and, so far, rejected.

9 — Two ways to map, and how to measure them

9.1 Tour versus exploration

The project can produce a map two ways. Both are kept, because they answer different questions.

Tour (frozen). `tour_node.py` drives 32 waypoints from `config/route.json` (each entry is `[x, y, 'drive'|'spin', note]`), publishing plain `Twist` commands — it does not use Nav2 at all. It takes roughly seven minutes of simulated time, produces a clean 359 × 278 map, and is the fastest way to a usable map for a known floor plan. It is also the phase-1 acceptance artefact: unchanged since 2026-10-20 17:21.

Frontier exploration. `frontier_explore.py` starts with no map, detects the boundary between free and unknown space on the live SLAM map, and asks Nav2 to drive to selected frontiers until none remain (or a goal/time budget is exhausted). It is the interesting one, and the rest of this chapter is about it — including the fact that, in this house, it never beat the tour map's coverage.

9.2 The exploration node, in the order it makes decisions

```
① detect      free cells adjacent to unknown → connected clusters (min_cluster = 12 cells)
              cluster centroid → candidate goal
② filter      reject goals closer than explore_min_goal_dist (0.60 m)
              reject goals whose clearance < explore_goal_clearance (0.40 m)
              reject goals in the "fake frontier" blacklist (see below)
③ pick        score by gain / distance, with a revisiting penalty (explore_revisit_gap 0.4)
④ pre-check   ask ComputePathToPose whether a path exists (timeout 1.5 s)
              no path → soft-blacklist for 30 s
              path exists but absurd → detour rule: dist < 3.0 m and path > 3 · dist + 1.5 m
              → soft-blacklist for 60 s
⑤ drive       send NavigateToPose; watchdog: no displacement for 12 s → abandon + blacklist
⑥ checkpoint  save map every 8 goals (this is what makes the coverage curve in §9.4 possible)
```

Two mechanisms deserve their own paragraph.

Fake frontiers. A frontier can persist for many goals because the planner keeps failing to reach it, which makes the node waste its entire budget on the same unreachable edge. The fix is a pending gain rule: a frontier whose expected gain is below `min_gain = 40` cells and which has been attempted (or pending) for more than 4 s is blacklisted, not retried. Measured effect: the "frontier ping-pong" pattern disappeared from the logs (Chapter 3 of the phase-3 notes).

Pre-checking with an action, not an assumption. Asking the planner for a path before sending the goal costs one action call (a few hundred milliseconds) and converts "drive there, get stuck, wait 12

s, blacklist" into "blacklist immediately". On a 40-goal run that is the difference between ~4 and ~11 stalls, depending on configuration.

9.3 What was actually compared

Three campaigns were run, each in its own isolated DDS domain and Gazebo partition, each writing to its own map prefix and CSV row. The driver is `tools/explore_ab.py` (§ 9.5).

(a) Planner A/B, 40 goals, identical start. The global planner was the variable.

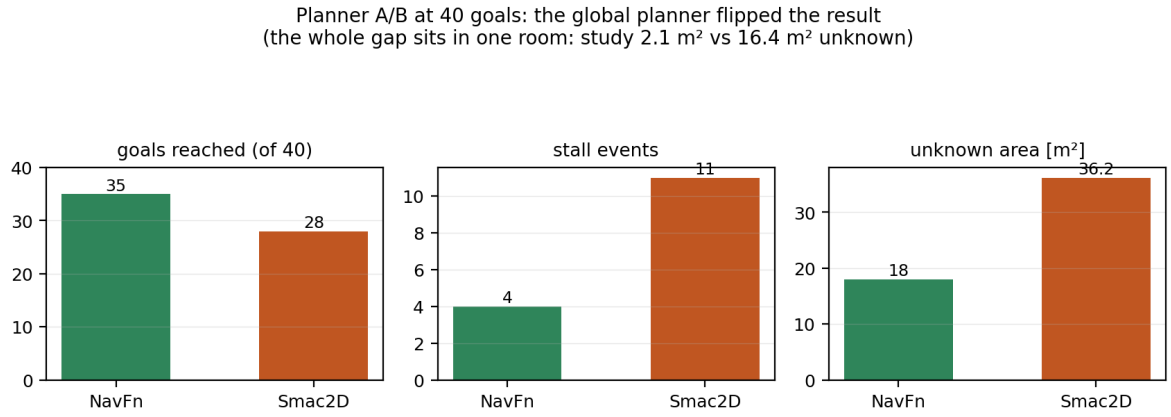


Figure 5: Planner A/B at 40 goals. The entire difference is concentrated in the study: 2.1 m^2 unknown with NavFn versus 16.4 m^2 with SmacPlanner2D.

Arm	reached	stalls	unknown	idle	sim time
NavFn	**35/40**	**4**	**18.0 m^2**	8.3 %	21.9 min
SmacPlanner2D	28/40	11	36.2 m^2	25.3 %	22.2 min

Verdict: NavFn stays. Note the reason the project believes this: it is not "NavFn is better" in general, it is that the difference is localised in one room, and that room's failures are driven by goal clearance (§ 8.2), which interacts with the planner's tolerance settings.

(b) Goal clearance 0.40 / 0.50 / 0.55, three runs each.

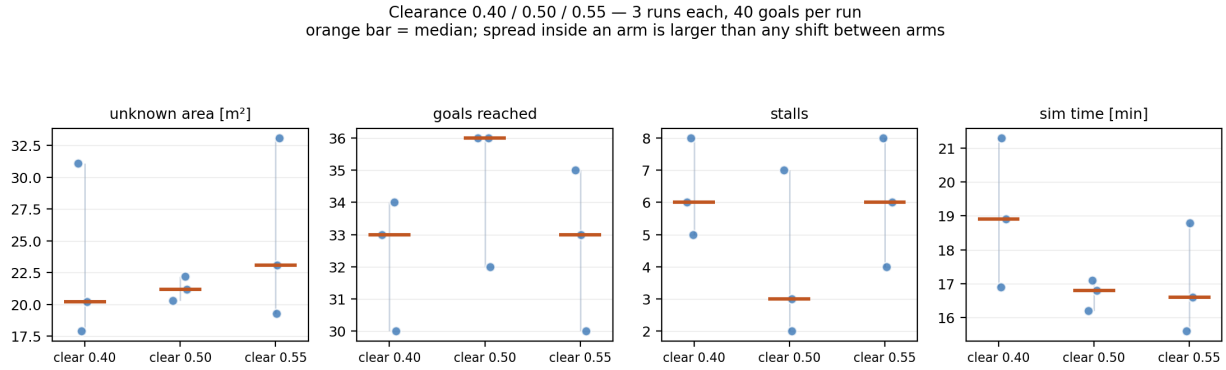


Figure 6: Three runs per arm, 40 goals each. Each dot is a run; the orange bar is the median. The spread inside an arm is larger than any shift between arms.

Arm	unknown (median [range])	reached	stalls	sim time	idle
0.40	20.2 [17.9–31.1]	33 [30–34]	6 [5–8]	18.9 [16.9–21.3]	11.3 %
0.50	21.2 [20.3–22.2]	36 [32–36]	3 [2–7]	16.8 [16.2–17.1]	9.6 %
0.55	23.1 [19.3–33.1]	33 [30–35]	6 [4–8]	16.6 [15.6–18.8]	10.6 %

Between-arm median differences versus within-arm ranges: unknown **2.9 vs 13.8 m²**, reached **3.0 vs 5.0**, stalls **3.0 vs 5.0**. Every between-arm difference is smaller than the noise inside an arm. The honest conclusion is therefore not "0.50 is best" — it is **"clearance is a safety floor, not a coverage knob; keep it ⩾ `inflation_radius` and stop tuning it."**

One of the nine runs was invalid (0 goals issued, 25 minutes, 182.8 m² unknown — a Nav2 lifecycle bring-up race, § 9.5) and was replaced by a retry on a new domain, whose result landed inside the 0.50 arm's range. That replacement is recorded, not hidden: replacing a failed run with a retry is legitimate, silently deleting it would not be.

(c) 80 goals: does more budget help?

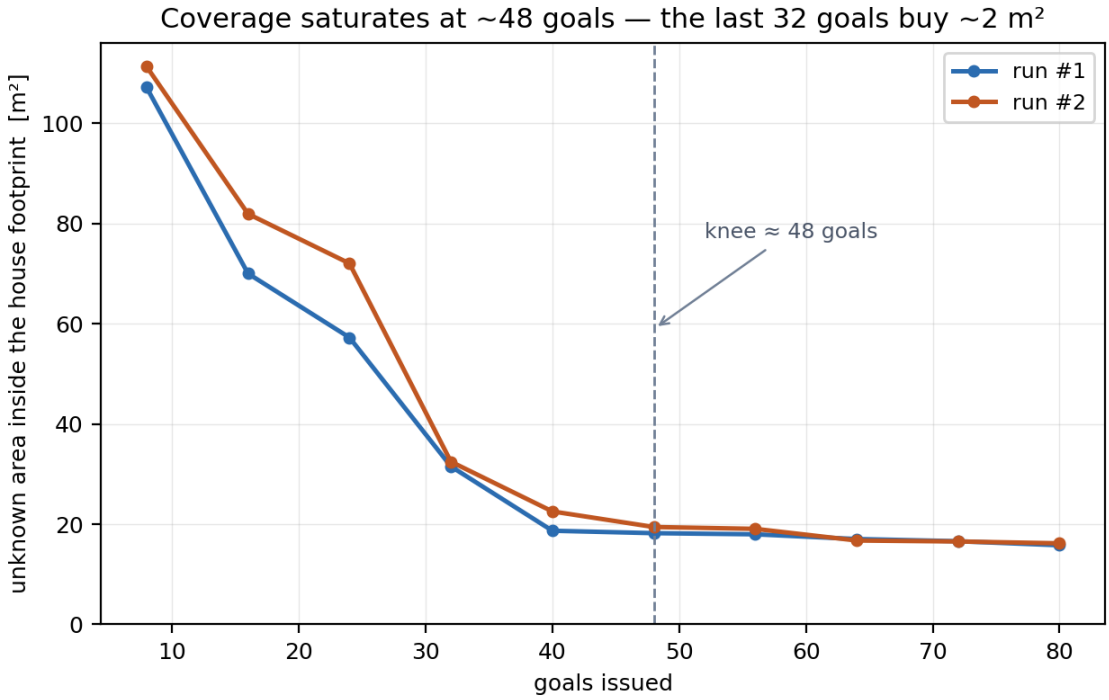


Figure 7: Coverage versus goals issued, recomputed from the checkpoint maps of both 80-goal runs.

goals	8	16	24	32	40	48	56	64	72	80
run 1	107.2	70.0	57.3	31.5	18.7	18.2	18.0	17.0	16.6	15.8
run 2	111.3	81.9	72.1	32.5	22.5	19.4	19.1	16.7	16.5	16.2

Both runs (63/80 and 57/80 reached, 40.9 and 40.7 minutes of simulated time) ended by hitting the goal cap, not by exhausting frontiers. The curve flattens around **48 goals**: the last 32 goals buy 2.2–2.9 m², at roughly 0.1 m² per goal, while stalls triple. Hence the project's budget rule: **choose the goal count at the knee of the curve (40–48), not a round number.**

9.4 Why no configuration beat the test map

The live acceptance map has 13.5 m^2 interior unknown. No exploration configuration ever beat it: the best 40-goal run left 15.8 m^2, the best 80-goal run 15.8 m^2. Two explanations, both partial:

- the 13.5 m^2 figure came from a single run of a different configuration and may itself be a lucky sample (it has never been reproduced under the current driver);
- the residue is dominated by shadow pockets behind furniture and unreachable slivers, which a frontier-based explorer cannot enter by construction.

The project records this as a **known ceiling**, not as a target: the remaining unknown area is not proportional to effort, and stating that plainly prevents a future maintainer from burning weeks on a metric that has already been measured to plateau.

9.5 The driver: how to run an experiment instead of a demo

`tools/explore_ab.py` (~420 lines) turns "run it a few times" into a procedure. Per arm and repetition it:

1. allocates a fresh `ROS_DOMAIN_ID` / `IGN_PARTITION` and a unique map prefix;
2. launches the stack and waits for the exploration started marker, with a **210 s boot self-check** that also fails on any `Failed to bring up` line from Nav2;
3. starts an evaluation probe that records velocity shares (idle / rotating / moving);
4. polls until "exploration finished — no frontiers" or the goal/time cap;
5. computes eight metrics itself (reusing `map_quality.py` for map decoding), appends a CSV row;
6. cleans up the domain and prints a median table.

Three validity guards exist because each was earned:

Guard	Failure it prevents	Evidence
210 s boot self-check + `nav_bringup_failed`	a run that issues 0 goals and burns 25 minutes producing a 182.8 m^2 "result"	the 0.50 #3 run
`no_goals` is never counted as complete	the same, in the aggregation stage	`explore_ab.py` filter
retry on a new domain, log name carries the domain	a retry overwriting the failed run's log — i.e. losing the only evidence of the failure	observed before the fix

Resume behaviour follows from the same principle: a run is "done" only if `status == done` **and** `arrived > 0`. Anything else is re-attempted.

9.6 Methodology: what this chapter cost

Three lessons, all recorded because they were learned expensively:

1. **One run per arm is not an experiment.** A 15-goal A/B showed SmacPlanner2D winning by 21 m^2; at 40 goals the same comparison reversed to an 18 m^2 loss. Short runs measure variance.
2. **Report ranges next to medians.** The 3×3 table is the reason the clearance question is closed: not because a winner was found, but because the noise floor was measured and every candidate effect fell below it.

3. **Budget by the knee, not by round numbers.** 80 goals is 40 minutes of simulation to buy 2 m^2 — an obviously bad trade once the curve exists, and a plausible plan before it did.

Takeaways. Tour and exploration coexist because they answer different questions. The exploration node' s value is in its cancellation machinery (fake frontiers, path pre-checks, stall watchdog), not in its frontier scoring. Three campaigns produced one adopted change (NavFn), one closed question (clearance) and one budget rule ($\approx$48 goals). Everything else was noise, and proving that it was noise was the work.

10 — The agent layer: language in, motion out

10.1 Four processes, two Python environments

The agent stack is split across four processes on purpose, and the split is the most common source of confusion for newcomers:

```
terminal 1   ros2 launch ...              system python 3.10 + rclpy     (simulation, SLAM, Nav2)
terminal 2   rosbridge_websocket          system python 3.10 + rclpy     (port 9090)
terminal 3   agent_bridge.py              system python 3.10 + rclpy     (/agent/* ⇄ Nav2)
terminal 4   mak_demo / main.py           project venv (Python 3.10)     (LLM, tools, semantic map)
```

The iron rule: **every process that imports `rclpy` must run on the system interpreter.** `rclpy` in ROS 2 Humble is compiled for the system Python; running it inside the project venv produces `ModuleNotFoundError: No module named 'rclpy._rclpy_pybind11'`, which looks like a broken ROS installation and is not. The agent side runs in the venv because it needs `agent-framework` and the OpenAI SDK, which are not system packages.

The two halves meet only over a websocket. That boundary is also a debugging boundary: if `/agent/pose` updates but the robot does not move, the problem is downstream (bridge or Nav2); if nothing updates, the problem is upstream (rosbridge or the venv side).

10.2 The `/agent/*` contract

Topic	Type	Direction	Content
`/agent/nav_goal`	`geometry_msgs/PoseStamped`	agent → bridge	target pose; sending a new one **supersedes** the old
`/agent/nav_status`	`std_msgs/String` (JSON, latched)	bridge → agent	state, progress, remaining distance, recovery count
`/agent/pose`	`geometry_msgs/PoseStamped`	bridge → agent	current pose at 5 Hz
`/agent/cancel`	`std_msgs/Empty`	agent → bridge	cancel the current goal

`/agent/nav_status` is **latched** (transient-local). This is not cosmetic: a status published by a daemon thread before any subscriber connects is otherwise lost forever, and the agent then waits for a state change that already happened. The same lesson applies to any status topic a tool subscribes to late.

`agent_bridge.py` (287 lines) owns one more thing: **goal sequencing**. Every goal sent gets a monotonically increasing `seq`, and all three callbacks (`on_goal_response`, `on_feedback`, `on_result`) carry it. Results whose `seq` is stale are logged and dropped:

```
已有目标在飞，先取消旧的（旧 #1 → 新 #2）
忽略已被取代的目标 #1 的结果（当前 #2）
```

Without this, Nav2's terminal result for the cancelled goal arrives after the new goal has started, and marks it `failed` — and worse, clears the handle that the cancel path and the watchdog need (Chapter 12 § 12.4, bug 1).

10.3 Models: measured, not evangelised

Model	Capabilities	Verdict
`qwen3-vl:8b`	completion + vision + tools + thinking	**default agent** — accurate Chinese, copies arguments correctly
`qwen2.5:7b`	completion + tools	**reporter only** — as a tool agent it emits fragmentary garbage (`"pora房间信息…"`)
`llama3.1:8b`	completion + tools	rejected: transcribed 洗手间 as 汉测式
`gemma3:12b`	completion + vision	rejected: no tool calling at all

The reporter split is a latency decision with a measurement behind it: rewriting a result into one sentence needs no reasoning, and `qwen2.5:7b` does it in **1.9 s** hot versus **31.7 s** for `qwen3-vl:8b` — a 15× difference that dominates every user-visible interaction. So the thinking model plans and calls tools; the fast model narrates.

Both endpoints are configurable (`--base-url`, `--api-key`, `ROBOT_LLM_*` environment variables), so moving to a cloud model is a flag change, not a refactor.

10.4 Why the model is not allowed to decide to move

The routing tree, implemented in code (`planner.py`, `preempt.py`, `agent_nav.py`):

```
utterance
  ├── classify: imperative (ACT) vs question (ASK)          — code, regex + verb table
  ├── ACT, resolvable target
  │     ├── "看看/确认" + object → mode D: look_at + VLM
  │     └── otherwise            → code sends the goal directly
  ├── ACT, unresolvable target  → code refuses, lists real places (no LLM call, no motion)
  ├── cancel words              → code sends /agent/cancel (route F), no LLM call
  └── ASK                       → LLM with tools: find_place / list_rooms / where_am_i
```

The reason is a measured rebellion, not a philosophy: given 「带我去沙发看看」, `qwen3-vl:8b` called `find_place`, answered "the sofa is in the middle of the living room", and stopped — the robot never moved. Adding prompt instructions and `tool_choice="required"` did not fix it, because the ollama compatibility layer ignores that field. For a robot standing in front of a user, "decided not to act" is indistinguishable from "broken".

So the contract is inverted: **code guarantees motion; the model handles what only it can do well** — resolving colloquial ambiguity, choosing among scored candidates, and phrasing the result as a sentence a person would say.

10.5 The tool surface

Seven tools are exposed to the LLM (`mak_demo/tools.py`, 292 lines):

Tool	Arguments	Effect
`list_rooms()`	—	names + one-line description of every room
`find_place(text, room='')`	Chinese phrase	scored candidates with reasons (no motion)
`where_am_i()`	—	current pose and the room it falls in
`go_to(object_id)`	resolved id	drives to the object's standoff pose
`go_to_room(room)`	room name	drives to room centre
`look_at(object_id)`	resolved id	drives to the observation pose, then VLM confirms
`check_path()`	—	laser + vision passability verdict (no motion)

Every tool returns a machine-readable status **and** a sentence, and every call is logged into a trace that is printed after the run. The trace is what makes "the robot said something odd" debuggable: you can see the exact arguments the model copied from the semantic map, which is where transcription errors (汉测式) surface.

10.6 Three self-checks after every run

`agent_nav.py` ends with an assertions block that has caught more defects than any unit test:

```
核查①  did the robot actually send motion commands, and did they succeed?
核查②  is the answer text clean (no English fragments, no leaked tool names)?
核查③  if a target was named, did the robot actually arrive?
```

Check ③ is the one that prevented the project from shipping a lying robot: after a preemption, an older build printed "arrived" for a goal that was cancelled. The check turns that into a loud `FAIL 有明确目标却没有成功到达 —— 任务未完成`.

> **Takeaways.** Four processes, one websocket, and a hard rule about which interpreter may import `rclpy`. The status topic is latched because late subscribers matter. Goals carry sequence numbers. The model plans and narrates; code moves the robot and refuses impossible requests. Self-checks assert arrival, not intent.

11 — The semantic map: a database, not a perception system

11.1 What it is

`config/semantics.json` (11 KB) contains eight room records (seven rooms plus the corridor) and 61 object records in 33 classes:

```
{ "id": "fridge_1", "class": "fridge", "room": "kitchen",
  "x": -2.66, "y": 5.20, "yaw": 1.571, "w": 0.78, "d": 0.78 }

{ "name": "living_room", "label_zh": "客厅",
  "rect": [-8.89, 1.56, -3.20, 6.89], "center": [-6.045, 4.225],
  "door_center": [-6.0, 1.5], "door_width": 1.3 }
```

It is **generated by `build_world.py` alongside the physics world**, and it is **read-only at runtime**: nothing in `mak_demo/` writes to it. The camera never contributes to it.

This is a deliberate engineering choice with a precise justification: the robot has a single monocular RGB camera (320 × 240, no depth channel) and the world is painted in flat colours with almost no texture. Detecting an object is possible; localising it in map coordinates is not, because a monocular image provides no scale and no range. A perception-built semantic map would therefore have been fabricated, not measured.

The trade it buys: object coordinates are exact to the millimetre, identical to the simulator's own placement table, so every navigation failure is a real failure rather than a data error. The trade it costs: the map cannot be discovered, only authored — a new environment means regenerating it, and moved furniture is invisible until the vision check at query time.

11.2 The lexicon, and how a phrase becomes a candidate

`mak_demo/data/lexicon.json` (35 entries) maps Chinese and English aliases to class names: fridge / 冰箱 / 雪柜, shoe cabinet / 鞋柜, and so on. Resolution (`semantic_map.resolve_object`) returns **scored candidates with reasons**, not a single answer:

```
"带我去那个放饮料的柜子"  → fridge_1  0.86  (alias 冰箱 + room context kitchen)
                          → wardrobe_1 0.31  (alias 柜子, wrong room)
```

Returning several candidates is what lets the LLM disambiguate with the user's own words, and what lets the code refuse cleanly when nothing scores above threshold (「家里没有叫「别了」的地方」 is the wrong behaviour of this mechanism, fixed in Chapter 12).

11.3 Standoff: where a robot should stand to interact with a thing

Object coordinates are centres; driving to a centre means driving into the object. The `standoff_pose()` computation is pure geometry — 16 directions sampled around the object, each candidate checked against the occupancy map, scored by clearance and by preference for facing the room centre:

```
CLEAR_M = 0.45 m   # must equal Nav2 inflation_radius — 0.30 m looked clean but produced
                   # "map says passable, physics says stuck" pockets between fridge and chairs
GAP_M   = 0.55 m   # standoff distance = object radius + 0.55 m
```

When no candidate satisfies the strict thresholds, the search relaxes in stages (`[0.45, 0.35]` → `[0.40, 0.25]` → `[0.32, 0.15]`), and the relaxation is recorded, because a relaxed standoff is a warning that the object is in a tight place — precisely the situation that produced the "irreversible squeeze" bug in phase 2A.

Two special cases:

- **Corridor landmarks.** Sixteen objects stand in the corridor but carry a room label (Chapter 3 § 3.4). `standoff_pose` uses the physical position; only the label is topological.
- **Objects with no free neighbour at all.** The resolver reports failure rather than returning a pose inside furniture; the agent then answers "I can't reach it from here" instead of driving.

11.4 The observation pose, and why 0.55 m fails

Seeing an object is a different problem from standing next to it. The observation pose is:

```
distance = object radius + 1.40 m        (fridge: 0.39 + 1.40 = 1.79 m from centre)
camera height = 0.70 m                   (measured, not assumed)
```

This pair was established by an A/B experiment with exactly one variable — camera height — and then by moving the distance, which turned out to matter more:

Camera height	Standoff 0.55 m (side of object)	Observation 1.40 m (frontal)
0.30 m	cannot identify objects; sees table legs and shelves	—
0.70 m	sees the side panel: VLM calls the fridge a **door**	**correct**: fridge recognised

The failure at 0.55 m is not a model problem. At that distance and a 1.204 rad horizontal field of view, the object fills the frame with one flat side; a front-facing model has nothing to recognise. The fix is geometric: back off and look at the front.

11.5 Mode D: vision confirmation with closed options

The VLM prompt is deliberately impoverished: a **closed list of all 28 object classes** plus a short answer format, and one question per frame. Open-ended prompting produced confabulation — an 8B model described kitchen floor tiles as bedsheets and called the fridge a door.

```
You see one photo from the robot's camera. Candidate classes: [28 names].
Answer in exactly this form: "The object is <class>." or "I cannot tell."
Do not describe the scene. Do not guess.
```

Measured: **13.1 s** warm (first call after model load is much slower), four checks green — the object was named, the frame exists on disk (`logs/look_*.png`), the distance matched the plan, and no other class was claimed.

Mode D is the only place the camera influences a decision, and even there its influence is limited to a yes/no about identity. It cannot move the robot, and it cannot write the map.

11.6 Mode E: passability, where the camera is genuinely useful

A 2D laser slice answers "is there a gap at 0.78 m?" but not "can this robot get through?" — an object below the slice, or a gap that is visible but too narrow for a 0.42 m body, both need context. Mode E therefore combines:

1. **Laser sector analysis**: nearest return in the forward sector, and whether a free corridor of at least the robot's width exists; reported as a tier ("blocked", "can advance 0.91 m", "clear").
2. **Vision second opinion**: if the laser says "can advance", ask the VLM whether the way looks physically blocked (a fridge filling the frame, a chair lying across the aisle).
3. **Disagreement is reported, not averaged.** A verdict of "laser says 1.12 m, vision says blocked" is more useful to a human than a weighted score.

The module also carries a scar: `MIN_R = 0.12 m`. When the lidar was mounted such that it saw its own mast (echoes at 0.124–0.138 m), that threshold passed them through and turned an open corridor into "obstacle at the nose" (Chapter 4 § 4.4d).

11.7 What the semantic map does not do

- It does not learn. Adding a room means regenerating from `build_world.py`.
- It does not detect a moved object. Mode D does, but only for the object you asked about.
- It does not know about anything that is not in the table (a box left in the corridor is only an obstacle, never a thing).
- Its labels are ownership labels, so a "in room X" query and a geometric point-in-polygon test will disagree for the sixteen corridor landmarks — by design.

Takeaways. The semantic layer is authored truth, read-only at runtime, with millimetre coordinates and no perception. The lexicon returns scored candidates so ambiguity can be resolved rather than guessed. Standing next to an object and seeing it are different geometric problems (0.55 m vs 1.79 m). The camera's only job in the pipeline is identity confirmation and a passability second opinion.

12 — Changing its mind: preemption

12.1 The interaction contract

The robot is expected to accept speech **while driving**. Three behaviours are required:

Utterance while driving	Expected behaviour
a different destination	abandon the current goal, drive to the new one, never claim the abandoned one arrived
「停下」 / “stop”	cancel, send no new goal, stop cleanly
a question	answer while continuing, and do not swallow the question

The third row is not decorative: it is the row that broke.

12.2 Judgment order is a code decision

`mak_demo/preempt.py` (101 lines) is a pure, testable table with a strict priority:

```
1. new destination present?  → PREEMPT   ("别去厨房了，去卧室" is a redirect, not a cancel)
2. question present?         → PARALLEL  ("停下停下冰箱在哪" must still answer the question)
3. cancel word/prefix only?  → CANCEL    ("停下" alone)
4. otherwise                 → NONE
```

Priority 2 above priority 3 is subtle and was learned the hard way: with the naive order, a pending “stop” in an utterance that also asked a question caused the question to be dropped entirely (Chapter 12 §12.4, bug 3). Priority 1 above 3 matters because “别去X了，去Y” contains a cancel-looking clause and a destination; treating it as a pure cancel strands the robot.

Segment filtering handles the compound case: `filter_cancel_segments()` drops comma-separated clauses that begin with 别去 / 不用去 / 不去 / 别往 ..., and returns the original list unchanged if everything was dropped (so a pure 「停下」 still reaches route F).

12.3 Three pieces, all required

Preemption is not a parsing problem. It is three concurrency changes, and removing any one degrades the system to “one command at a time”:

Piece	Without it
every tool call runs in `asyncio.to_thread`	the event loop blocks inside `go_to`, so `input()` never returns and nobody can speak at all
one `asyncio.Task` per utterance	a new utterance waits for the previous task to finish — the definition of not preempting
code-side judgment table	the model decides, and re-decides inconsistently under prompt changes

The first row was measured, not assumed. With a synchronous tool call, a heartbeat coroutine fired **0** times in a 0.5 s window; wrapped in `to_thread` it fired **25** times.

12.4 Four bugs that only a live run could find

All four passed the offline suite. All four appeared within minutes of a real session.

Bug 1 — the new goal reports failure in 0.1 s. Symptom: after a preemption, the new goal printed `failed` after 0.1 s while the robot was still driving, and the agent answered "could not reach the living room, maybe an obstacle". Root cause: `agent_bridge.py` had no goal identity. Nav2's terminal result for the cancelled goal (status 5) arrived after the new goal had started, was written into the new goal's state, and cleared `goal_handle` — so the cancel path and the watchdog could no longer reach the real goal. Fix: a monotonic `seq` per goal, incremented **before** the old handle is released, carried through response/feedback/result callbacks, with stale callbacks logged and dropped. Guardrail: the offline suite greps the source for `self.goal_seq += 1` before the release and for all three callbacks taking a sequence argument.

Bug 2 — cancellation stops nothing. Symptom: after 「停下」 the terminal kept printing `剩余 5.28 m` every two seconds for two minutes until the 180 s timeout. Root cause: cancelling an `asyncio` task cannot interrupt a thread already inside `to_thread`. The worker kept polling a frozen link. Fix: a registry of in-flight goals in `nav_client.py` with `stop()` / `stop_all()`; the polling loop checks the flag and returns `('canceled', {'detail': 'agent preemption'})` immediately. `canceled` is deliberately distinct from `rejected`. Guardrail: a unit test that starts a fake `go_to` and asserts it returns `canceled` within 2 s.

Bug 3 — three utterances become one. Symptom: typing three lines produced the single string `停下停下冰箱在哪`, and the question was swallowed as a cancel. Root cause: the progress printer ran in a worker thread and wrote to the same terminal as `input()`, interleaving prompts and swallowing newlines. Fix: progress printing is off by default in interactive mode (`--progress` re-enables it); plus the judgment order of §12.2. Guardrail: a test asserting that a concatenated stop+question string routes to PARALLEL.

Bug 4 — the cancel clause becomes a place name. Symptom: 「别去了，去客厅」 produced `FAIL 家里没有叫「别了」的地方，这一步跳过`. The behaviour was eventually right, but the user saw a nonsensical rejection. Root cause: the segmenter split on the comma and fed the first clause to the resolver. Fix: `CANCEL_PREFIX` + `filter_cancel_segments()`. Guardrail: four segment-level test cases.

The pattern across all four: **the offline tests verified the logic I wrote, and the live run verified the logic I did not write** — lifecycle ordering, thread ownership, terminal interleaving, and segmentation.

12.5 Acceptance evidence

```
t = 0 s    「带我去厨房」              → drives
t = 15 s   「别去了，去客厅」           → PREEMPT; bridge logs "旧 #1 → 新 #2"
```

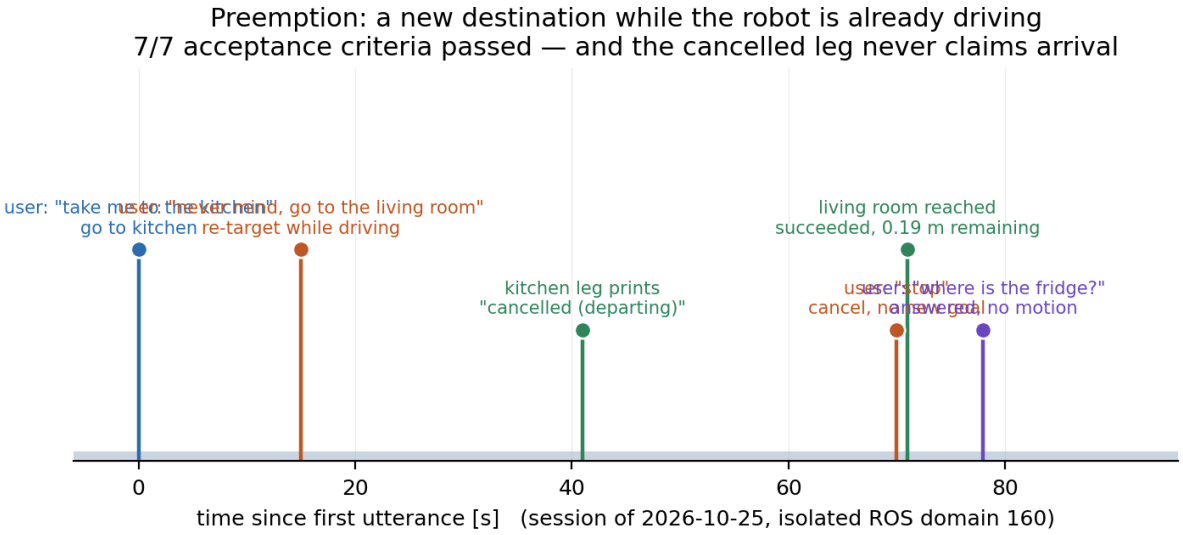


Figure 8: The accepted session, isolated ROS domain 160. The cancelled leg prints "cancelled (departing)" and never claims arrival.

```
t = 41 s  cancelled leg prints      STOP 「带我去厨房」已取消（发车）
t = 71 s  living room               PASS succeeded, 26.9 s, 0.19 m remaining
t = 70 s  「停下」                   → CANCEL, no new goal
t = 78 s  「冰箱在哪」                → PARALLEL, "fridge is in the kitchen, east, ~3.5 m"
```

Seven criteria, all measured: preemption announced; cancelled leg says "cancelled" and not "arrived"; new goal `succeeded` (not `failed`); no spurious `failed`; no residual progress spam after the stop; the cancel clause is not resolved as a place; no false arrival claim.

Because a manual session cannot reliably hit "the robot is mid-corridor at t = 15 s", the run is driven by a script (`preempt_live.py`) that feeds utterances at fixed times through the interactive REPL. That is the same reproducibility principle as Chapter 9, applied to a conversation.

12.6 What remains fragile

- **One input stream.** The REPL is the only input; there is no voice, no wake word, no barge-in. A second input source would need the same task-per-utterance discipline.
- **Cancellation is cooperative.** It works because every blocking call checks a flag. A new blocking call added later must do the same, or bug 2 returns in a new costume.
- **Status is a JSON string.** Adequate, but a typed message plus a sequence field would remove the possibility of parsing drift between the two languages (Python and rclpy) that share it.

Takeaways. Preemption is concurrency, not parsing: `to_thread`, one task per utterance, and a code-side priority table — all three or nothing. Every goal needs an identity, or stale results rewrite live state. The four real bugs were all invisible to offline tests, which is why the acceptance procedure includes a scripted live session.

13 — How we know it works

13.1 Test layers

Layer	Cost	What it can catch	What it cannot
Offline suite (11 checks)	seconds, no ROS	parsing, planning tables, geometry, source-level guardrails	anything about ROS lifecycle or timing
Unit checks for the scan guard (6 cases)	milliseconds	judging logic for `/scan` publisher counts	a real duplicate publisher
Runtime gates G1–G8	~6 minutes with simulation	TF, bridging, Nav2, agent, vision, refusal, compound commands	long-run degradation
Campaigns (Chapter 9)	20–45 minutes per run	statistical effects on coverage and stability	anything semantic
Scripted live sessions	minutes	concurrency, preemption, terminal interaction	visual quality

The order matters: cheap layers run first every time, and the expensive layer is reserved for questions the cheap ones cannot answer.

13.2 The offline suite

```
$ cd mak_demo && .venv/bin/python dev/check_*.py        # 11/11
```

Check	Asserts
`check_checks`	the post-run assertion block itself behaves correctly
`check_intent`	ACT/ASK classification over real utterances
`check_tools`	tool argument handling and refusal paths
`check_refusal`	unresolvable target refuses without motion
`check_planner`	multi-clause plans and their ordering
`check_passability` (19 cases)	laser sector tiers, self-echo exclusion
`check_vlm_guard` (3 cases)	VLM answer validation, closed-option parsing
`check_vision_options`	the 28-class option list is complete and stable

Check	Asserts
`check_look_route` (9 cases)	observation-pose selection and failure reporting
`check_standoffs`	standoff geometry against the real semantic map
`check_preempt` (40 assertions, 5 groups)	judgment table, segmentation, fake-link cancellation, `to_thread` heartbeat, **source-level guardrails**

Two things are unusual and worth copying:

- **`check_preempt` reads the source.** It greps `agent_bridge.py` for `self.goal_seq += 1` before the handle release, for the three callbacks accepting a sequence, and for `navc.VERBOSE` being set from `--progress`. A behavioural test cannot see "the ordering is wrong"; a source assertion can, cheaply.
- **The suite runs without ROS, ollama or a simulator.** That is why it can be run on every edit, which is why it finds things.

13.3 The eight runtime gates

Each gate is a script; each prints a verdict and exits non-zero on failure, so the whole ladder can be run unattended.

```
# terminal 1 — the stack (isolated domain)
ROS_DOMAIN_ID=161 IGN_PARTITION=ab161 ros2 launch ./dan_house.launch.py \
  lidar:=3d nav:=true loc:=truth headless:=true \
  map:=config/maps/dan_house_lighthouse3d.yaml

# terminals 2–3 — bridge path (system python, both)
ros2 launch rosbridge_server rosbridge_websocket_launch.xml
/usr/bin/python3 agent_bridge.py

# terminal 4 — gates
cd mak_demo
.venv/bin/python dev/check_link.py            # G1
.venv/bin/python dev/check_loc.py             # G1b
.venv/bin/python dev/test_nav.py              # G2
.venv/bin/python dev/test_semantic_nav.py 冰箱 # G3
.venv/bin/python agent_nav.py "带我去冰箱看看"    # G4
.venv/bin/python agent_nav.py "前面能过去吗"     # G5
.venv/bin/python agent_nav.py "洗手间在哪"       # G6
.venv/bin/python agent_nav.py "带我去地下室"     # G7
.venv/bin/python main.py "先去客厅，然后到冰箱旁边" # G8
```

Baseline versus last full run (2026-10-26, domain 161):

Gate	Baseline	Last run	Verdict
G1 link	25 pose frames, status idle	24 frames, status idle	pass
G1b localisation	0.00 m / 0.0°	0.00 m / 0.0°	pass

Gate	Baseline	Last run	Verdict
G2 drive (3.0, 0.0)	57.0 s, 0.197 m residual	13.3 s, 0.19 m residual	pass (faster: different start pose)
G3 semantic 冰箱	—	63.0 s, 0.15 m off plan	pass
G4 mode D	1.79 m, VLM 58.5 s	1.79 m, VLM 13.1 s warm	pass
G5 mode E	—	laser 1.12 m, vision “blocked”, agree	pass
G6 mode B	41.7 s, “south ~9.2 m”	41.1 s, 9.2 m	pass
G7 refusal	0.0 s	0.0 s	pass
G8 compound	91.6 + 58.0 s	55.6 + 58.2 s	pass

The G8 result is the one that mattered most in the last regression: the fix for bug 4 of Chapter 12 touched exactly the segment-filtering path that compound commands use, so a passing G8 was the evidence that the fix did not damage the older behaviour.

13.4 Isolation, or why the numbers can be trusted

Every measurement in this book was produced with:

```
ROS_DOMAIN_ID = fresh value per run (87–104, 120–129, 130, 140, 150/151, 160, 161 ...)
IGN_PARTITION = matching value (lhL104, ab120 ... ab161)
map prefix    = tag_arm_rep (e.g. ..._ab_1025_2047_0.40_1)
log name      = tag_arm_rep_d{domain}.log
```

Consequences: no run can read or overwrite another’s map; a failed run’s log is never overwritten by its retry (the domain is in the filename); and after the fact, every number in the CSV can be traced to one log and one map prefix.

13.5 The scan-guard scenarios (reproducible in five minutes)

```
3D normal            PASS  /scan publisher 1: /pointcloud_to_laserscan
3D negative          FAIL  /scan has 2 publishers: /_ros2cli_12764, /pointcloud_to_laserscan
    (inject the second publisher:)
    ros2 topic pub /scan sensor_msgs/msg/LaserScan "{header: {frame_id: base_scan},
      angle_min: -3.14159, angle_max: 3.14159, angle_increment: 0.01745,
      range_min: 0.12, range_max: 12.0, ranges: [1.0]}" -r 5
rogue publisher gone PASS  recovered without relaunch
2D normal            PASS  /scan publisher 1: /gz_bridge
```

13.6 Six rules of evidence used throughout

1. **Verify the effect, not the call.** “The publisher exists” is not “the subscriber received it”; status topics are latched for this reason, and gates assert arrival, not intent.
2. **Numbers carry a domain and a map prefix.** An unattributable number is a rumour.
3. **A failed run is recorded, not deleted.** It is the only evidence of the failure mode.
4. **A single run supports no claim.** Chapter 9 § 9.6.
5. **Distinguish “loading” from “broken”.** The 210 s boot self-check, and preflight’s 45-second heuristic.

6. **A cancellation must never look like an arrival.** This one sentence summarises half of Chapter 12.

   Takeaways. Five test layers, cheapest first. The offline suite is unusual in that it reads the source for ordering invariants. Gates are scripts that exit non-zero. Every number is traceable to a domain, a map prefix and a log.

14 — Delivery: what is frozen, what is open

14.1 The lineage of every artefact

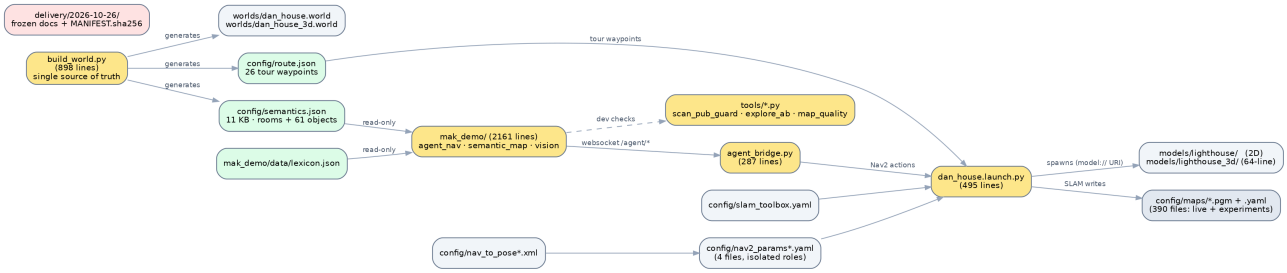


Figure 9: Generated, consumed, frozen. Yellow boxes are code; green boxes are generated data; the red box is the read-only delivery snapshot.

Reading the figure left to right is the fastest way to understand the repository:

Layer	Artefacts	Authority
Generator	`build_world.py` (898 lines)	the only place world truth is written
Generated data	`worlds/*.world`, `config/semantics.json`, `config/route.json`	never edited by hand
Models	`models/lighthouse/`, `models/lighthouse_3d/`	spawned via `model://` URIs
Runtime	`dan_house.launch.py` (495 lines) + `config/*.yaml`, `config/nav_to_pose*.xml`	wiring order and parameters
Evidence	`config/maps/*` (390 files), `experiments/ab_explore.csv`, logs	append-only in spirit
Agent	`mak_demo/` (2161 lines), `agent_bridge.py` (287)	language, semantics, concurrency
Tools	`tools/*.py`	measurement, assertions, drivers
Frozen	`delivery/2026-10-26/`	read-only, hash-manifested

14.2 The frozen snapshot

Five documents were frozen on 2026-10-26 as the deliverable set:

```
delivery/2026-10-26/
  README.md                       1281 lines   full project manual (Chinese)
  STATUS.md                       644 lines   acceptance state + chapters ①–⑮ of experiments (Chinese)
  docs/phase2A_results.md          372 lines   agent-guided navigation results (Chinese)
  docs/phase2B_vision.md           705 lines   camera / VLM / passability results (Chinese)
  docs/phase3_lidar3d.md           437 lines   64-line lidar conversion (Chinese)
  MANIFEST.sha256                              SHA-256 of all five
  交付说明.md                                    index: scope, acceptance, gaps, how to verify
```

Properties that make it a freeze rather than a copy:

- byte-identical to the working files at the moment of freezing (`cmp` verified);
- file mode 444 — an accidental edit announces itself;
- `sha256sum -c MANIFEST.sha256` verifies all five;
- the working documents carry a pointer to the snapshot, so a reader who edits them knows a frozen version exists;
- the snapshot records the git state (`main`, HEAD `45dd84e`, 1570 uncommitted paths) and states explicitly that it freezes **working-tree content, not a commit**.

That last point is the snapshot's main weakness and is stated in the index: to reproduce from source control, these files must be committed and the resulting hash recorded.

14.3 Reproducing the system from zero

```
# 1. the world (regenerates world + semantics + route)
python3 build_world.py --lidar 3d

# 2. mapping the classical way (frozen tour)
ros2 launch ./dan_house.launch.py lidar:=3d slam:=true tour:=true headless:=true

# 3. save the map (during or after the tour)
ros2 run nav2_map_server map_saver_cli -f config/maps/my_map

# 4. navigate + agent
ros2 launch ./dan_house.launch.py lidar:=3d nav:=true loc:=truth headless:=true \
  map:=config/maps/dan_house_lighthouse3d.yaml
#   terminal 2: ros2 launch rosbridge_server rosbridge_websocket_launch.xml
#   terminal 3: /usr/bin/python3 agent_bridge.py
#   terminal 4: cd mak_demo && .venv/bin/python main.py --interactive

# 5. exploration instead of a tour
ros2 launch ./dan_house.launch.py lidar:=3d slam:=true explore:=true nav:=true headless:=true
```

Two operational rules that cost hours when ignored: stop the simulation with **Ctrl+C in its terminal** (not `pkill`), and run `bash preflight.sh` afterwards until it reports zero gz processes. An orphaned physics engine is invisible in `ros2 topic list` and expensive in CPU.

14.4 What is deliberately not done

This list is the most important part of the book for a future maintainer, because every item is a decision, not an oversight.

Gap	Status	Why it matters
`loc:=amcl` (real-robot localisation)	**not re-verified** since truth poses were adopted	every acceptance number in this book assumes truth poses; hardware has none
EKF fusion drift	measured 7.1 m / −37° over a long run, unsolved	blocks any "wheel odometry only" configuration
3D obstacles in the costmap	not implemented; all four param files read `LaserScan`	overhangs, table tops and shelves are invisible to navigation
Semantic map from perception	not attempted	monocular camera without depth; the map is authored truth instead
Coverage beyond 13.5 m^2 interior unknown	measured ceiling	frontier exploration plateaus; the residue is shadow pockets
Voice input / wake word	not implemented	the REPL is the only input channel
Warehouse-scale behaviour	analysed, not built	Chapter 15
Real hardware	never run	this is a simulation project throughout

14.5 Reading order for a newcomer

1. This book's Chapter 1 (ten minutes) — what the system does.
2. `README.md` §0–§5 — the hands-on path to a map and a goal.
3. `STATUS.md` "验收状态" table — what is claimed, with evidence links.
4. `docs/phase2A_results.md` §3 — why authority moved out of the LLM.
5. Chapter 9 of this book — how experiments are run here, before proposing a change.

Takeaways. One generator writes all world truth. Evidence is append-only and traceable. The delivery snapshot is verifiable and honest about being a working-tree freeze. Chapter 14.4 is the project's conscience: eight things it does not do, each with a reason.

15 — From a house to a warehouse

A natural next question — and the one that decides whether this architecture generalises — is what it would take to replace the seven-room house with an Amazon-style fulfilment centre. This chapter answers it with the code as the unit of accounting, and it is deliberately written before any warehouse code exists, so the estimate can be checked later.

15.1 What is world-specific today

Only five things in the repository know anything about this house:

Artefact	Size	Warehouse equivalent
`build_world.py` — room table, corridor constant, door widths, route table	898 lines, hardcoded table	a new generator (racks, aisles, docks, charging bays)
`worlds/*.world`	165 lines, 136 models (generated)	regenerated
`config/semantics.json`	61 objects, 33 classes (generated)	racks, pallets, totes, conveyor, dock doors
`config/route.json`	32 waypoints (generated)	inspection / pick route
`mak_demo/data/lexicon.json`	35 alias entries	warehouse vocabulary + slot addresses
`config/nav2_params*.yaml`	tuned for 16 m rooms	retuned for metre-wide aisles at 60 m scale

Everything else — 3900 lines of robot model, launch wiring, agent layer, tools, experiments — is world-agnostic as written.

15.2 What is hardcoded that would have to change

Two structural couplings, both small and both worth naming precisely:

(a) The launch file cannot choose a world. `dan_house.launch.py` holds `WORLD_2D` and `WORLD_3D` as constants and selects between them only by `lidar`. A warehouse needs a `world:=home|warehouse` argument resolving to a bundle {`world_file, semantics, lexicon, map_prefix, params`} — roughly 20 lines.

(b) The agent resolves paths, not places. `semantic_map.py` reads `config/semantics.json`, `mak_demo/data/lexicon.json` and one map path as module constants. A warehouse needs a configuration root parameter (~30 lines), plus the test vocabulary, which is spread across ~100 occurrences of home-specific words in `mak_demo/dev/check_*.py`.

15.3 Three changes that are architectural, not configurational

This is where the real work is. They are listed in order of how deeply they cut.

1. Scale. The house is 17.8 × 13.8 m with a 60 m loop. A fulfilment centre is 60 × 40 m or larger with kilometre-scale driving. Consequences: laser range (12 m) truncates in open dock areas; SLAM loop closure over long aisle loops is untested here (the recorded drift experiment only covers the house); the global costmap grows to ~1200 × 800 cells (still tractable, but update rates must be re-measured); and the exploration budget rule must be re-derived — the coverage knee at 48 goals is a property of this world's frontier density, not a law.

2. 3D obstacles become mandatory. This is the change that cannot be deferred. A warehouse is full of geometry that a single horizontal slice at one height cannot see: rack beams, pallets at varying heights, forklift tines, conveyor overhangs, and the gap under a shelf that a 0.85 m tall robot could enter but never leave. Today all four Nav2 parameter files declare `observation_sources: scan` with `data_type: "LaserScan"`. The 64-line point cloud is already published and bridged but **nothing subscribes to it**. The costmap must be fed from `/scan/points` (voxel layer with a real 3D source, or several horizontal slices with explicit heights), and the passability logic must stop assuming a plane.

3. Semantics change shape, not just vocabulary. Today a place is `room + object(x, y, yaw, w, d)`. A warehouse's central concept is a **slot address** — aisle-rack-bay-level, e.g. `A-12-03-2` — which is a coordinate system, not an object list. The resolver therefore needs a new module (`slot_map.py`, ~200–300 lines) that parses an address, expands rack geometry into accessible slots, and reuses `standoff_pose()` for the final pose (because "drive to a rack face, not into it" is exactly the geometry already solved here).

Two secondary consequences: the lexicon must map operator language to slot addresses ("A12 货架第三个托盘"), and mode D's visual confirmation becomes more valuable, not less — it is the only mechanism that can check "am I in front of the rack the address claims".

15.4 Effort, in self-contained steps

Step	Deliverable	Estimate	Acceptance
W1	warehouse world generator + first map	1–2 days	SDF valid, tour/exploration produces a map, no collisions
W2	parameter adaptation (Nav2, exploration, route) + large-scale localisation	0.5–1 day	gates G1/G1b/G2 pass in the new world
W3	slot semantics + vocabulary + agent integration	1–2 days	"go to A-12-03-2" drives and arrives; mode D confirms the rack
W4	3D obstacles in the costmap	1–2 days	a pallet above the slice is avoided, not driven under

Total 4–6 days, each step independently acceptable with the existing gates — which is the real argument for the architecture: **the acceptance machinery transfers unchanged**, so the estimate is bounded by world-building and parameter work, not by redesign.

15.5 The recommendation recorded here

Add a world module inside this repository rather than forking the project:

- ~3900 lines are world-independent; a fork would mean fixing every future bug twice (the four preemption bugs of Chapter 12, and the scan guard, would each have two homes);
- the phase-1 baseline is frozen as a read-only delivery snapshot, so new work must be additive by construction;
- isolation is already available per world (separate semantics/route/maps directories, separate `world:=` value, separate branch).

Shape of the change:

```
build_world_warehouse.py          new generator (the existing 898 lines untouched)
worlds/warehouse_{2d,3d}.world    generated
config/warehouse/{semantics.json, route.json, nav2_params*.yaml, maps/}
mak_demo/data/lexicon_warehouse.json
dan_house.launch.py               + world:=home|warehouse  (~20 lines)
branch: feature/warehouse-world
```

15.6 Decision

As of 2026-10-26 this work is **deliberately postponed**. The current project's objectives are met and frozen; a warehouse would add two days of world-building before the interesting question (3D obstacles, slot semantics) can even be posed. Recording the analysis now means the decision can be re-opened cheaply later, with the estimate already on the table.

> **Takeaways.** The world is data; the stack is mechanism. Two small couplings (world selection, configuration root) stand between the current code and a second environment. Three changes are architectural: scale, 3D obstacles, slot-address semantics. The acceptance machinery is the part that transfers for free — and the part that makes a 4–6 day estimate credible.

Appendix A — Topics and nodes

A.1 Topics

Topic	Type	Producer	Consumers
/clock	rosgraph_msgs/Clock	gz bridge	everything (use_sim_time:=true)
/odom	nav_msgs/Odometry	gz bridge	odom_tf_node, Nav2
/odom_wheels	nav_msgs/Odometry	gz bridge	EKF experiments
/imu	sensor_msgs/Imu	gz bridge	EKF experiments
/joint_states	sensor_msgs/JointState	gz bridge	robot_state_publisher
/cmd_vel	geometry_msgs/Twist	Nav2, tour, teleop	gz bridge
/scan	sensor_msgs/LaserScan	bridge (2D) / pointcloud_to_laserscan (3D)	slam_toolbox, costmaps, exploration, passability, scan guard
/scan/points	sensor_msgs/PointCloud2	gz bridge (3D only)	(nothing, today — Chapter 14 § 14.4)
/camera/image_raw	sensor_msgs/Image	gz bridge	mak_demo/vision.py, camera dev checks
/camera/camera_info	sensor_msgs/CameraInfo	gz bridge	camera dev checks
/map	nav_msgs/OccupancyGrid	slam_toolbox or map_server	Nav2, exploration, standoff checks
/tf, /tf_static	tf2_msgs/TFMessage	slam_toolbox, odom_tf_node/EKF, robot_state_publisher, static publishers	everything
/agent/nav_goal	geometry_msgs/PoseStamped	agent (websocket → bridge)	agent_bridge
/agent/nav_status	std_msgs/String (JSON, latched)	agent_bridge	agent
/agent/pose	geometry_msgs/PoseStamped	agent_bridge (5 Hz)	agent
/agent/cancel	std_msgs/Empty	agent	agent_bridge

A.2 Nodes

Node	Package / file	Role
gz_bridge	ros_gz_bridge/parameter_bridge	the ten-topic contract of Chapter 5 § 5.3
pointcloud_to_laserscan	pointcloud_to_laserscan	3D → 2D projection (3D mode only)
robot_state_publisher	robot_state_publisher	URDF → static and dynamic TF
odom_tf_node	project	integrates ground-truth pose into odom→base_footprint
ekf_node	robot_localization	alternative odom→base_footprint (ekf:=true)
slam_toolbox	slam_toolbox/async_slam_toolbox_node	mapping and map→odom
map_server, lifecycle_manager	nav2_map_server, nav2_lifecycle_manager	map loading when loc:=truth
controller_server, planner_server, bt_navigator, behavior_server	nav2_*	navigation
tour_node	project (521 lines, frozen)	32-waypoint tour publisher
frontier_explore	project (943 lines)	exploration
scan_pub_guard	tools/scan_pub_guard.py	/scan single-publisher assertion
rosbridge_websocket	rosbridge_server	DDS ⇄ websocket (:9090)
agent_bridge	agent_bridge.py (287 lines)	/agent/* ⇄ Nav2 actions, goal sequencing
agent side	mak_demo/main.py, agent_nav.py	REPL, routing, LLM tools, preemption

Appendix B — Parameter reference

B.1 Localisation and TF

Parameter	Value	File
`loc`	`amcl` | `truth`	launch
`ekf`	`false` | `true`	launch
`transform_publish_period`	0.02 s	`slam_toolbox.yaml`

B.2 SLAM

Parameter	Value
`mode`	`mapping`
`scan_topic`	`/scan`
`resolution`	0.05 m
`map_update_interval`	2.0 s
`min_laser_range` / `max_laser_range`	0.12 / 12.0 m
`minimum_travel_distance`	0.3 m
`minimum_travel_heading`	0.3 rad
`use_scan_matching`	true

B.3 Scan projection (3D mode)

Parameter	Value
`target_frame`	`base_scan`
`min_height` / `max_height`	−0.75 / +0.05 m
`angle_min` / `angle_max`	$\pm\pi$
`angle_increment`	0.0174533 rad (360 rays)
`range_min` / `range_max`	0.12 / 12.0 m
`use_inf`	true

B.4 Navigation

Parameter	`nav2_params.yaml` (mapping)	`nav2_params_nav.yaml` (acceptance)
`max_vel_x`	0.15	0.26
`min_vel_x`	0	−0.15
`robot_radius`	0.23	0.23
`inflation_radius`	0.45	0.45
`cost_scaling_factor`	3.0	3.0
`progress_checker`	0.25 m / 6 s	0.25 m / 6 s
`xy_goal_tolerance`	0.20 m	0.20 m
planner	NavFn	NavFn
controller	DWB (`FollowPath`)	DWB
`observation_sources`	`scan` (LaserScan)	`scan` (LaserScan)

B.5 Exploration

Parameter	Default	Meaning
`explore_max_goals`	120	hard cap
`explore_max_minutes`	60.0	wall/sim cap
`explore_min_cluster`	12	frontier cells to be a candidate
`explore_revisit_gap`	0.4	penalty when re-targeting nearby
`explore_goal_clearance`	0.40	must be $\geqslant$ `inflation_radius` in practice
`explore_min_goal_dist`	0.60	skip too-close frontiers
`explore_stall_secs`	12.0	watchdog (18 s measured worse)
`min_gain`	40 cells	fake-frontier blacklist threshold
plan pre-check	`ComputePathToPose`, 1.5 s	reject unreachable goals early
soft-blacklist TTL	30 s (no path) / 60 s (detour)	

B.6 Agent layer

Item	Value
LLM endpoint	`http://127.0.0.1:11434/v1` (ollama, OpenAI-compatible)
Agent model	`qwen3-vl:8b`
Reporter model	`qwen2.5:7b` (1.9 s vs 31.7 s hot)
Framework	`agent-framework` 1.22.0
rosbridge	`ws://127.0.0.1:9090`
Standoff gap / clearance	0.55 m / 0.45 m
Observation distance	object radius + 1.40 m
Camera height	0.70 m
Passability `MIN_R`	0.12 m

Appendix C — Glossary (Chinese ↔ English)

中文	English	Where
巡游 / 航点	tour / waypoint	`tour_node.py`, `route.json`
建图	mapping	Chapter 7
探索 / 边界	frontier exploration	`frontier_explore.py`
卡死	stall	watchdog, § 8.4
假边界	fake frontier	§ 9.2
膨胀半径 / 膨胀税区	inflation radius / inflation tax region	§ 8.2
站位	standoff pose	§ 11.3
观察站位	observation pose	§ 11.4
语义地图	semantic map	Chapter 11
词表 / 别名	lexicon / alias	§ 11.2
可通行性	passability	§ 11.6
抢占 / 中途改主意	preemption	Chapter 12
叫停词	cancel keyword	§ 12.2
判定表	decision table	§ 12.2
序号（目标序号）	goal sequence number	§ 10.2
验收闸门	acceptance gate	Chapter 13
离线套件	offline suite	§ 13.2
真值定位	truth-pose localisation	§ 7.4
冻结 / 交付快照	freeze / delivery snapshot	§ 14.2
货位 / 槽位地址	slot address	§ 15.3

Appendix D — File index

Approximately 10,000 lines of Python, measured at delivery time.

File	Lines	Purpose
`frontier_explore.py`	943	exploration node (Chapter 9)
`build_world.py`	898	world + semantics + route generator (Chapter 3)
`tour_node.py`	521	frozen tour publisher (Chapter 9)
`dan_house.launch.py`	495	runtime wiring (Chapter 5)
`tools/explore_ab.py`	454	experiment driver (§ 9.5)
`mak_demo/agent_nav.py`	422	intent routing, modes A/D/E/F/ASK (§ 10.4)
`mak_demo/main.py`	323	REPL, preemption, `--progress` (Chapter 12)
`mak_demo/tools.py`	292	the seven LLM tools (§ 10.5)
`agent_bridge.py`	287	`/agent/*` ⇄ Nav2, goal sequencing (§ 10.2)
`mak_demo/dev/check_preempt.py`	258	40 assertions incl. source guardrails (§ 13.2)
`mak_demo/semantic_map.py`	256	resolve / standoff (Chapter 11)
`mak_demo/vision.py`	230	camera capture, VLM prompt, closed options (§ 11.5)
`tools/explore_progress.py`	194	read-only progress viewer
`tools/map_compare.py`	181	map comparison / scoring
`mak_demo/passability.py`	155	laser sector tiers (§ 11.6)
`mak_demo/nav_client.py`	143	Nav2 client, in-flight registry, `stop_all` (Chapter 12)
`tools/scan_pub_guard.py`	140	`/scan` assertion (§ 6.4)
`mak_demo/rosbridge_link.py`	139	websocket transport
`tools/map_quality.py`	123	the single PGM decoder (§ 7.2)
`gen_textures.py`	120	procedural floor/wall textures
`mak_demo/preempt.py`	101	judgment table, segmentation (§ 12.2)
`odom_tf_node.py`	78	`odom→base_footprint` from truth pose (§ 4.3)
`mak_demo/planner.py`	57	multi-clause splitting
`mak_demo/bearing.py`	43	relative direction phrasing

D.2 Configuration and data

Path	Size	Notes
`config/semantics.json`	11 KB	8 rooms, 61 objects, 33 classes (generated)
`config/route.json`	2.1 KB	32 waypoints (generated)
`mak_demo/data/lexicon.json`	3.3 KB	35 alias entries
`config/slam_toolbox.yaml`	—	Chapter 7
`config/nav2_params*.yaml`	—	four roles (§ 8.1)
`config/nav_to_pose_explore.xml`	48 lines	exploration behaviour tree
`config/maps/`	390 files	live + experimental + preserved maps
`experiments/ab_explore.csv`	17 columns	every campaign run, valid or not
`docs/book/`	—	this book’ s sources and figures

Appendix E — Experiment log

Tag	Question	Arms	Runs	Verdict
—	goal pre-check (§ 9.2)	off / on	—	adopted: converts stalls into immediate blacklists
Recovery a	BackUp-first BT	stock / backup-first	—	rejected: 37.6 vs 45.0 m^2, 44 % slower
—	stall watchdog	12 s / 18 s	—	kept 12 s: 18 s gave no benefit (4 vs 2 stalls, worse coverage)
—	planner c	NavFn / SmacPlanner2D	40 goals	NavFn adopted: 35/40 vs 28/40, 18.0 vs 36.2 m^2
`1025_1647`	clearance	0.40 / 0.50 / 0.55	3 × 3, 40 goals	no effect above noise → clearance is a floor, not a knob
`1025_2047`	budget	0.40	2 × 80 goals	knee at ≈48 goals; 80 goals buys 2 m^2 in 41 min
`gc55`	clearance 0.55, single run	0.55	1	superseded by `1025_1647` (single-run artefact)
`smoke`, `smoke2`	driver validation	0.40	2	driver works; one `no_goals` run invalidated a policy

The full per-run rows (domain, map prefix, log path, eight metrics) are in `experiments/ab_explore.csv`; every row can be traced to a log file and a map prefix (Chapter 13 § 13.4).